

Test und Zuverlässigkeit

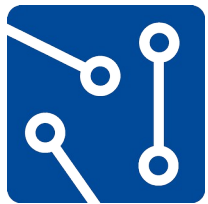
Test and Reliability

Summer Term 2026

Prof. Dr. Ralf Wimmer

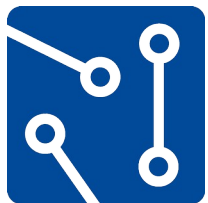
Dr. Tobias Seufert

Chapter 6: BIST and Test Compression



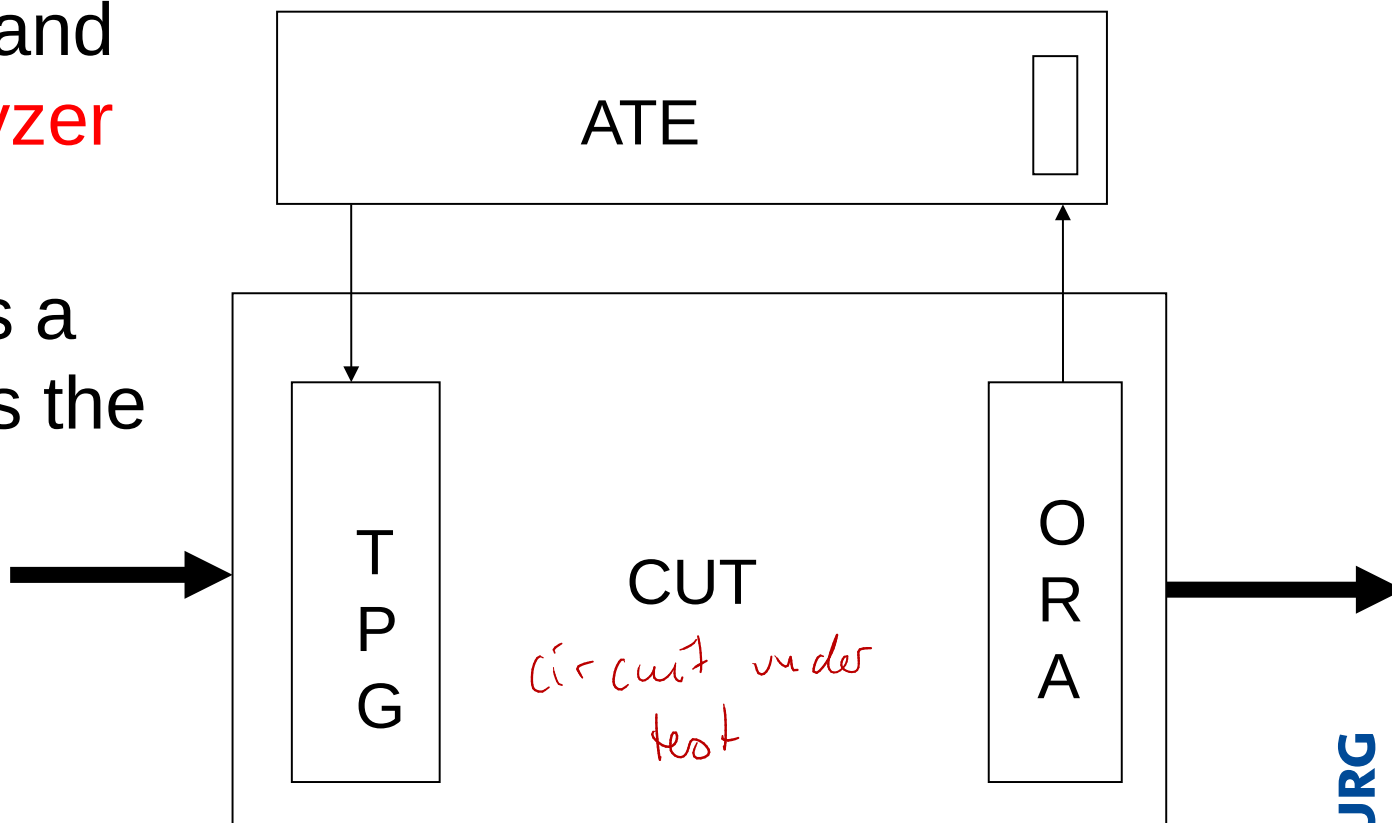
Outline

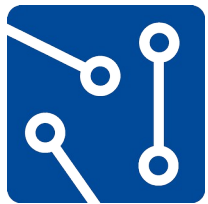
- Introduction: Types of BIST
- Exhaustive and pseudoexhaustive test
- On-chip Test Pattern Generation
 - Linear Feedback Shift Register (LFSR)
 - Weighted Random Pattern Generation
 - LFSR Reseeding
 - Bit flipping / bit fixing
- Output Compaction
- Test Compression



BIST Hardware

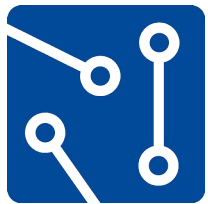
- BIST hardware includes a **test pattern generator** (TPG) and an **output response analyzer** (ORA)
- The external tester sends a start signal, and observes the ORA results after test application is complete





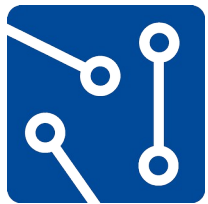
The Need for BIST

- The amount of **memory** required on a tester is **reduced**
 - No need to store test vectors or test responses
- BIST hardware can be placed “close” to the CUT
 - Large circuits contain embedded modules that are **difficult to access**
 - Creating sensitized paths is difficult
 - Access through scan registers is **time consuming**
 - BIST eliminates the need to scan values through long register chains
- BIST allows circuits to be **tested at-speed**
 - Circuit speeds tend to be **higher than tester speeds**
 - BIST hardware is implemented using the **same technology** as the CUT
 - At-speed testing is important for **delay defects**



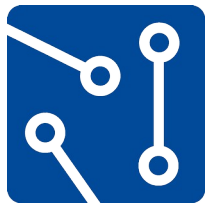
Different types of BIST

- **On-line BIST:** Testing occurs during normal functional operating conditions
 - **Concurrent on-line BIST:** Testing occurs simultaneously with normal functional operation
 - **Non-concurrent on-line BIST:** Testing occurs while the circuit is in an idle state
- **Off-line BIST:** Testing occurs when the system is not carrying out its normal functions



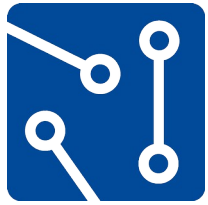
Concurrent BIST Implementations

- **Coding techniques:** Based on sets of lines that can only carry specific combinations.
 - Example: A **parity bit** added to every word stored in a memory. If the parity bit does not match the memory word, an error is identified.
- **Duplication and comparison:** Based on using multiple copies of the same circuit to do the same computation. If the output values do not agree, an error is identified.
 - Example: **Triple Modular Redundancy** (TMR), see Realibility chapter



Concurrent vs. Off-Line BIST

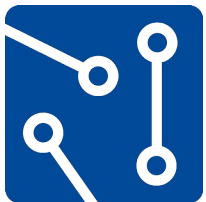
- Concurrent BIST identifies errors **when they first occur**, or shortly after they occur. With off-line BIST errors will be identified during a test session.
- Off-line BIST allows more **thorough testing** since it is not limited to sequences that occur during normal operation.



Off-Line BIST: Types of TPGs

Test Pattern Generators

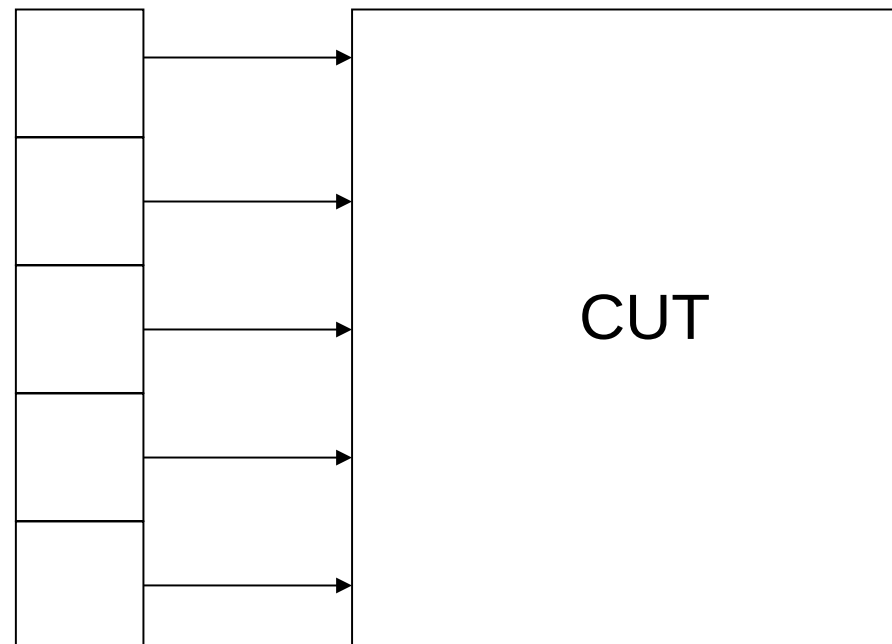
- Exhaustive and pseudo-exhaustive
- Pseudo-random and weighted pseudo-random
- Circuit-specific TPGs

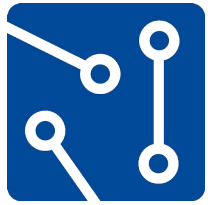


Exhaustive Testing of a Combinational Circuit

Binary counter goes through 2^n input vectors, where n is the number of CUT inputs.

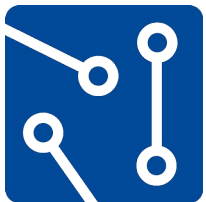
The circuit is tested exhaustively under all its input vectors 00000, 00001, 00010, ..., 11111



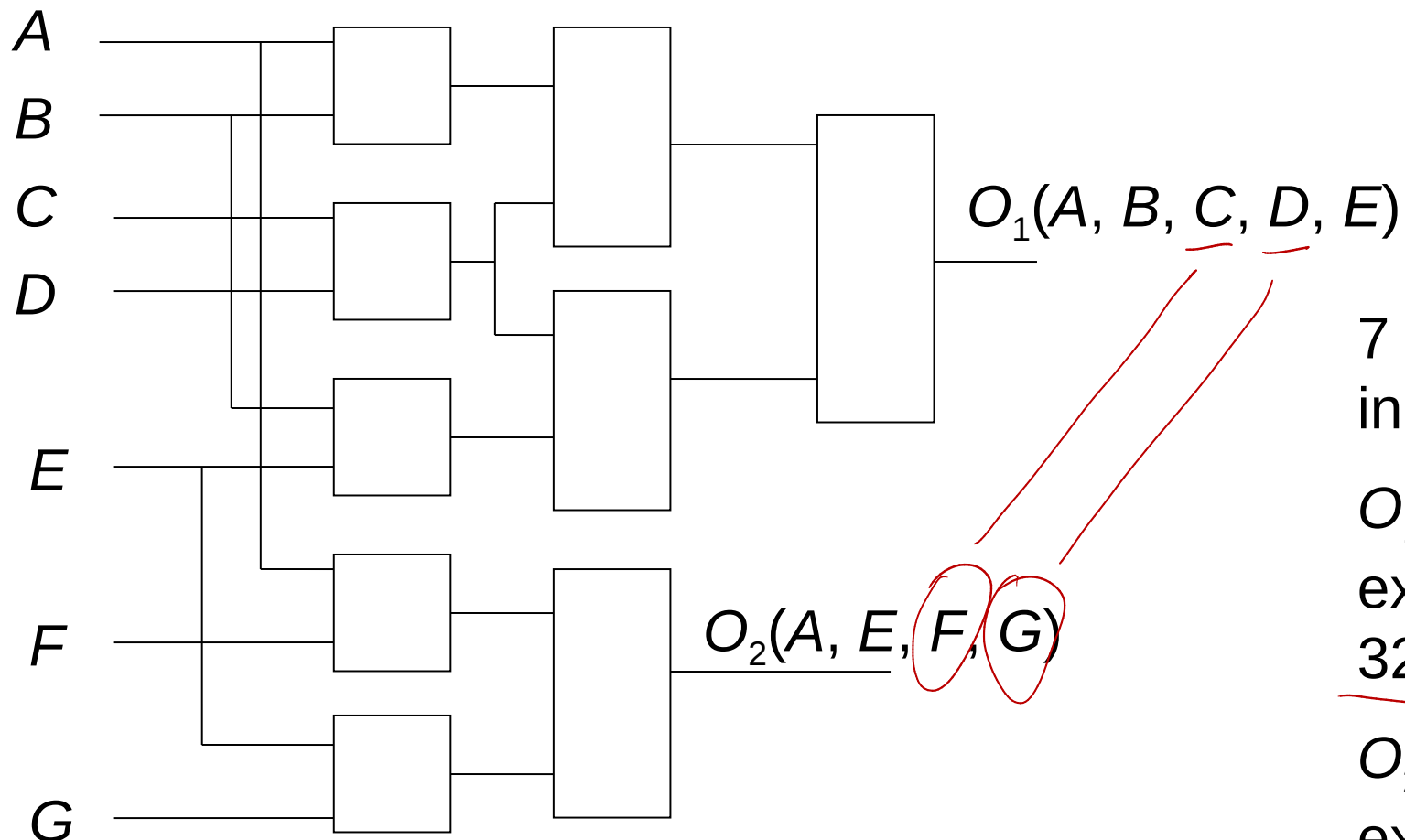


Pseudo-Exhaustive Testing

- When n is large, it is **not practical** to apply 2^n input vectors
- **Pseudo-exhaustive** testing partitions the circuit into segments, and tests each segment exhaustively
- **Segmentation** is based on the fact that primary outputs do not depend on all the primary inputs



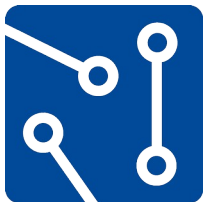
Example



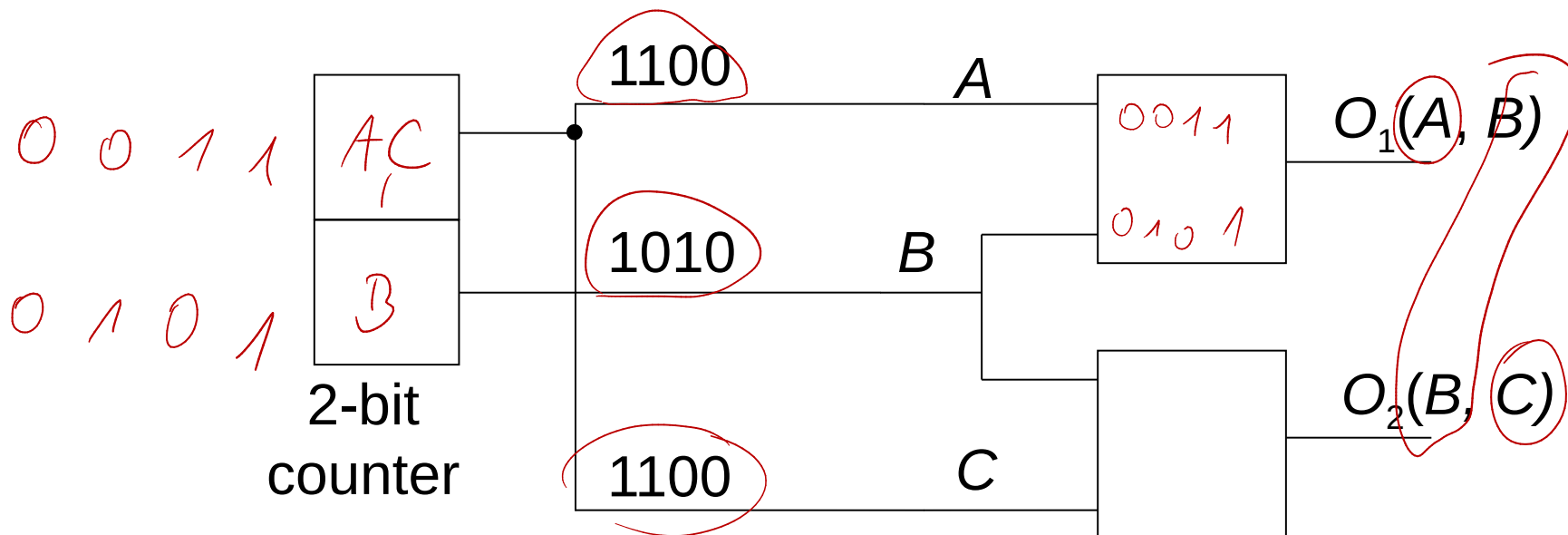
7 inputs – 128
input vectors.

O_1 can be tested
exhaustively using
32 vectors.

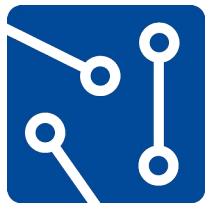
O_2 can be tested
exhaustively using
16 vectors.



Test Signal Inputs

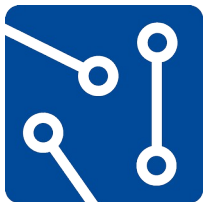


A and C can be driven from the **same cell** of a binary counter. They are considered the same test signal input. B requires an additional test signal input.



The Importance of Test Signal Inputs

- p – number of test signal inputs. $2^p \geq 2^w$
- w – maximum number of inputs on which any output depends.
- Number of pseudo-exhaustive tests is between 2^w and 2^p .
- If all outputs can be tested simultaneously ($p = w$): 2^w is obtained
- The goal is to use a **single binary counter** and connect it appropriately according to the test signal inputs.

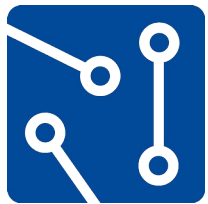


Computing p

- Step 1: Generate a dependency matrix.

<u>a</u>		<u>$f_1(a, b, e)$</u>
<u>b</u>		<u>$f_2(b, c, g)$</u>
<u>c</u>		<u>$f_3(a, d, e)$</u>
<u>d</u>		<u>$f_4(c, d, e)$</u>
<u>e</u>		<u>$f_5(e, f)$</u>
<u>f</u>		
<u>g</u>		

	a	b	c	d	e	f	g
f_1	$\times$	$\times$			$\times$		
f_2		$\times$	$\times$				$\times$
f_3	$\times$			$\times$	$\times$		
f_4			$\times$	$\times$	$\times$		
f_5					$\times$	$\times$	

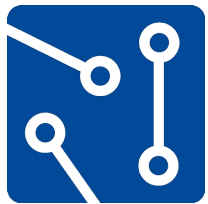


Computing p

- Step 1: Generate a dependency matrix.

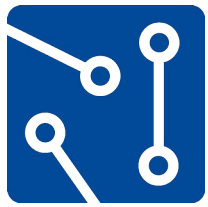
a		$f_1(a, b, e)$
b		$f_2(b, c, g)$
c		$f_3(a, d, e)$
d		$f_4(c, d, e)$
e		$f_5(e, f)$
f		
g		

	a	b	c	d	e	f	g
f_1	1	1	0	0	1	0	0
f_2	0	1	1	0	0	0	1
f_3	1	0	0	1	1	0	0
f_4	0	0	1	1	1	0	0
f_5	0	0	0	0	1	1	0



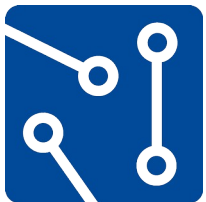
Step 2 – Partitioning

- Partition the columns of the dependency matrix into **groups** such that the inputs in the same group do not affect the same output.
 - Inputs in a group can be assigned the same values.
 - The counter will apply all the input combinations over the groups.
- Find pairs of inputs that can be **grouped together**. Combine groups into larger groups if possible.



Grouping the Inputs

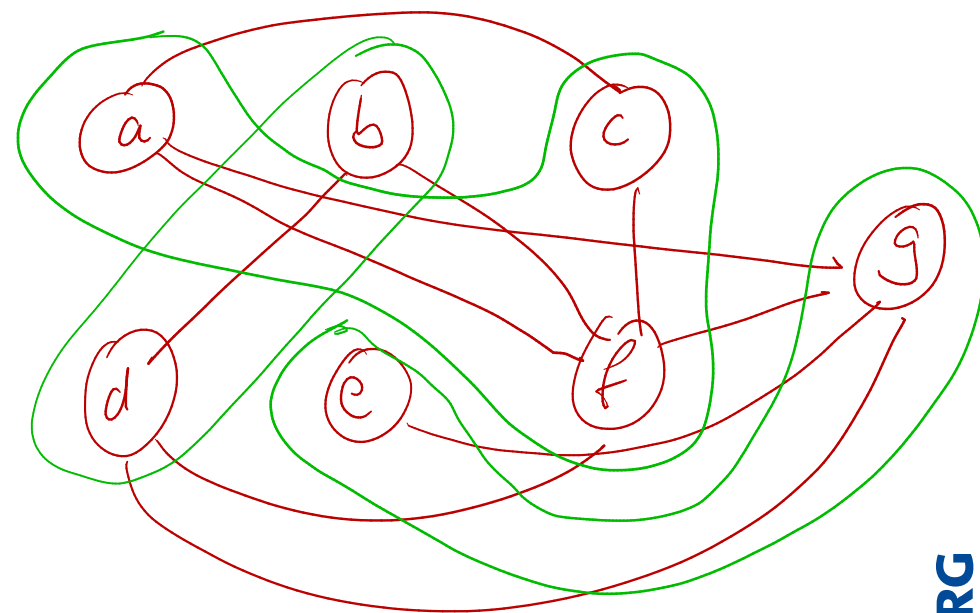
	a	b	c	d	e	f	g
f_1	1	1	0	0	1	0	0
f_2	0	1	1	0	0	0	1
f_3	1	0	0	1	1	0	0
f_4	0	0	1	1	1	0	0
f_5	0	0	0	0	1	1	0

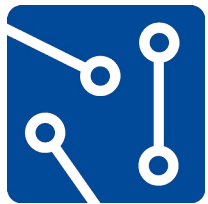


Grouping the Inputs

	<i>a</i>	<i>b</i>	<i>c</i>	<i>d</i>	<i>e</i>	<i>f</i>	<i>g</i>
f_1	<u>1</u>	1	<u>0</u>	0	1	<u>0</u>	0
f_2	<u>0</u>	1	<u>1</u>	0	0	<u>0</u>	1
f_3	<u>1</u>	0	<u>0</u>	1	1	0	0
f_4	<u>0</u>	0	<u>1</u>	1	1	0	0
f_5	<u>0</u>	0	<u>0</u>	0	1	<u>1</u>	0

a,c *a,f* *a,g* *b,d* *b,f*
c,f *d,f* *d,g* *e,g* *f,g*





Grouping the Inputs

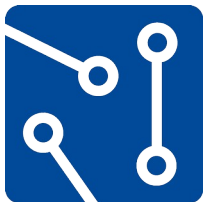
	a	b	c	d	e	f	g
f_1	1	1	0	0	1	0	0
f_2	0	1	1	0	0	0	1
f_3	1	0	0	1	1	0	0
f_4	0	0	1	1	1	0	0
f_5	0	0	0	0	1	1	0

a, c a, f a, g b, d b, f
 c, f d, f d, g e, g f, g

$a, c; a, f; c, f \rightarrow a, c, f$

$a, f; a, g; f, g \rightarrow a, f, g$

etc.



Grouping the Inputs

	a	b	c	d	e	f	g
f_1	1	1	0	0	1	0	0
f_2	0	1	1	0	0	0	1
f_3	1	0	0	1	1	0	0
f_4	0	0	1	1	1	0	0
f_5	0	0	0	0	1	1	0

a,c a,f a,g b,d b,f
 c,f d,f d,g e,g f,g

$a,c; a,f; c,f \rightarrow a,c,f$

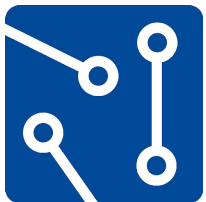
$a,f; a,g; f,g \rightarrow a,f,g$

etc.

Use:

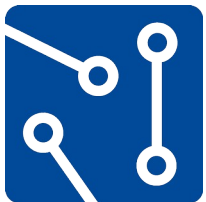
$a,c,f; b,d; e,g$

See Test Set Compaction (Coverage with Cliques)!



Using $\{a,c,f\}$; $\{b,d\}$; $\{e,g\}$

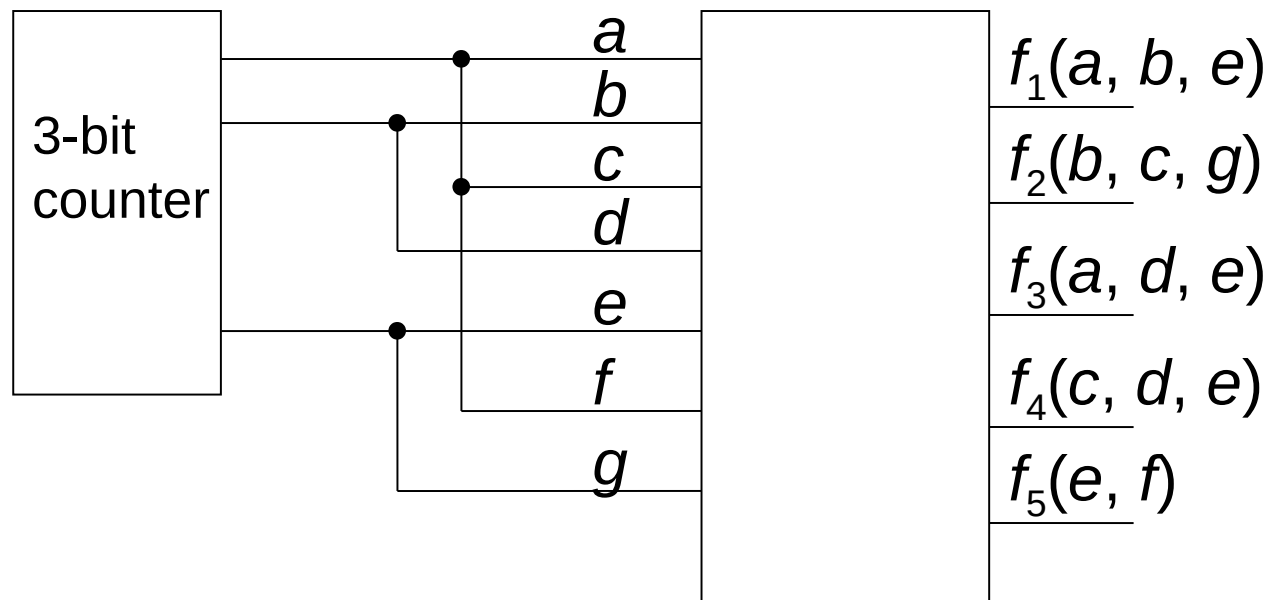
	a	c	f		b	d		e	g
f_1	1	0	0		1	0		1	0
f_2	0	1	0		1	0		0	1
f_3	1	0	0		0	1		1	0
f_4	0	1	0		0	1		1	0
f_5	0	0	1		0	0		1	0

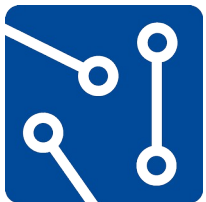


Test Generation using Pseudo-Exhaustive Testing

1 bit for each clique

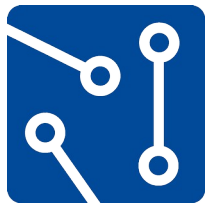
bit 0	a, c, f
bit 1	b, d
bit 2	e, g





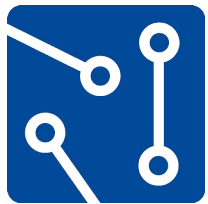
Test Vectors

<i>a</i>	<i>c</i>	<i>f</i>		<i>b</i>	<i>d</i>		<i>e</i>	<i>g</i>
0	0	0		0	0		0	0
0	0	0		0	0		1	1
0	0	0		1	1		0	0
0	0	0		1	1		1	1
1	1	1		0	0		0	0
1	1	1		0	0		1	1
1	1	1		1	1		0	0
1	1	1		1	1		1	1



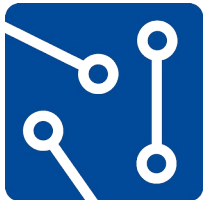
Extension of the Test Input Concept

- Use deterministic ATPG
- Find pairs of inputs such that if the inputs are forced to have the same value, deterministic ATPG can achieve **complete fault coverage**
- Extend to **maximal subsets** of inputs.
- Use a counter with a number of bits equal to the number of input subsets



Pseudo-random TPGs

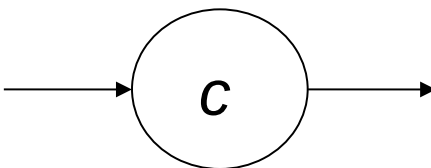
- Generate test vectors that have **characteristics of random patterns**
 - Approximately equal numbers of 0s and 1s on every input
- The vectors are generated **deterministically** and are therefore **repeatable**
- Pseudo-random test vectors without replacement can be generated by a **Linear Feedback Shift Register (LFSR)**

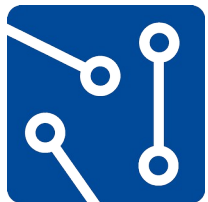


Linear Feedback Shift Registers (LFSRs)

■ A linear circuit is constructed from:

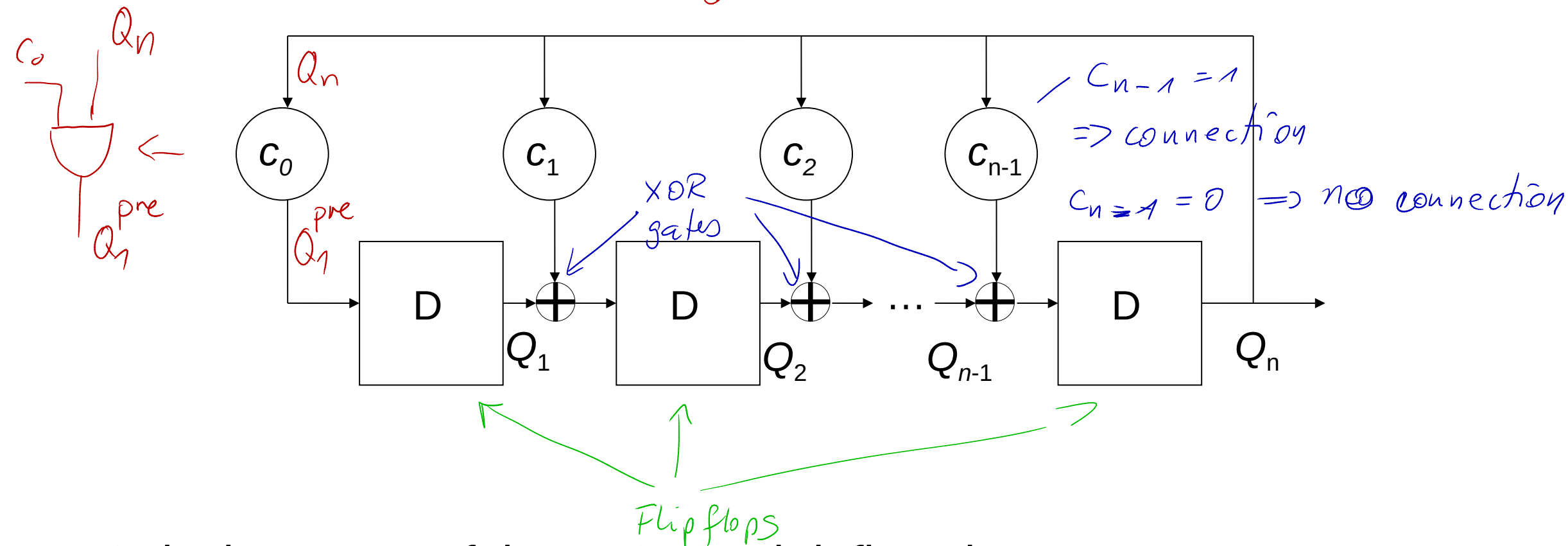
- Unit delays (D-FFs)
- Modulo-2 adders (XOR gates)
- Modulo-2 scalar multipliers ($c = 1$ implies a connection, $c = 0$ implies no connection)



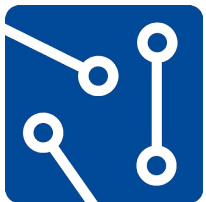


An LFSR

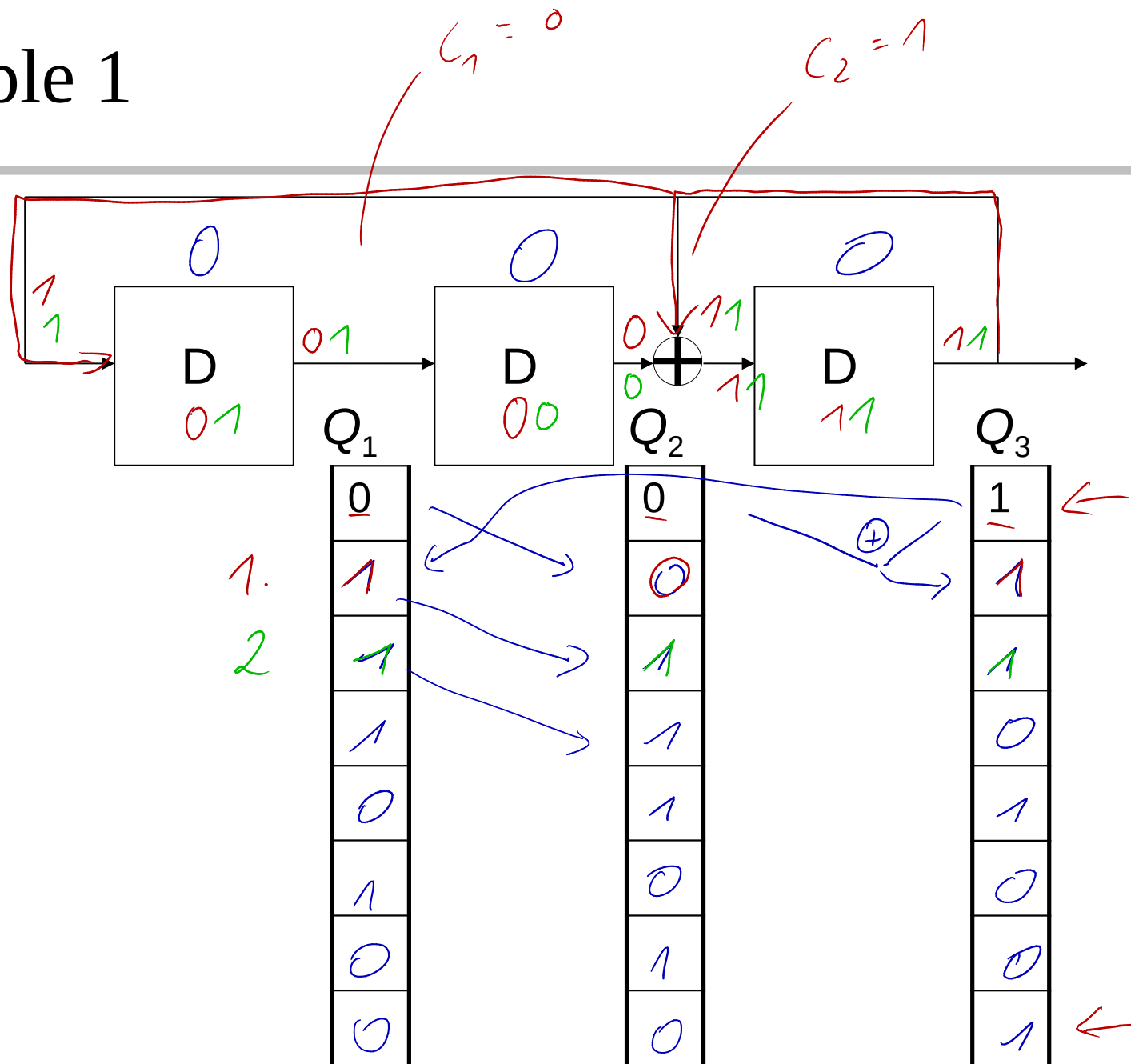
Galois LFSR

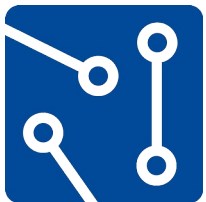


Q_n is the output of the LFSR and defines its output sequence.

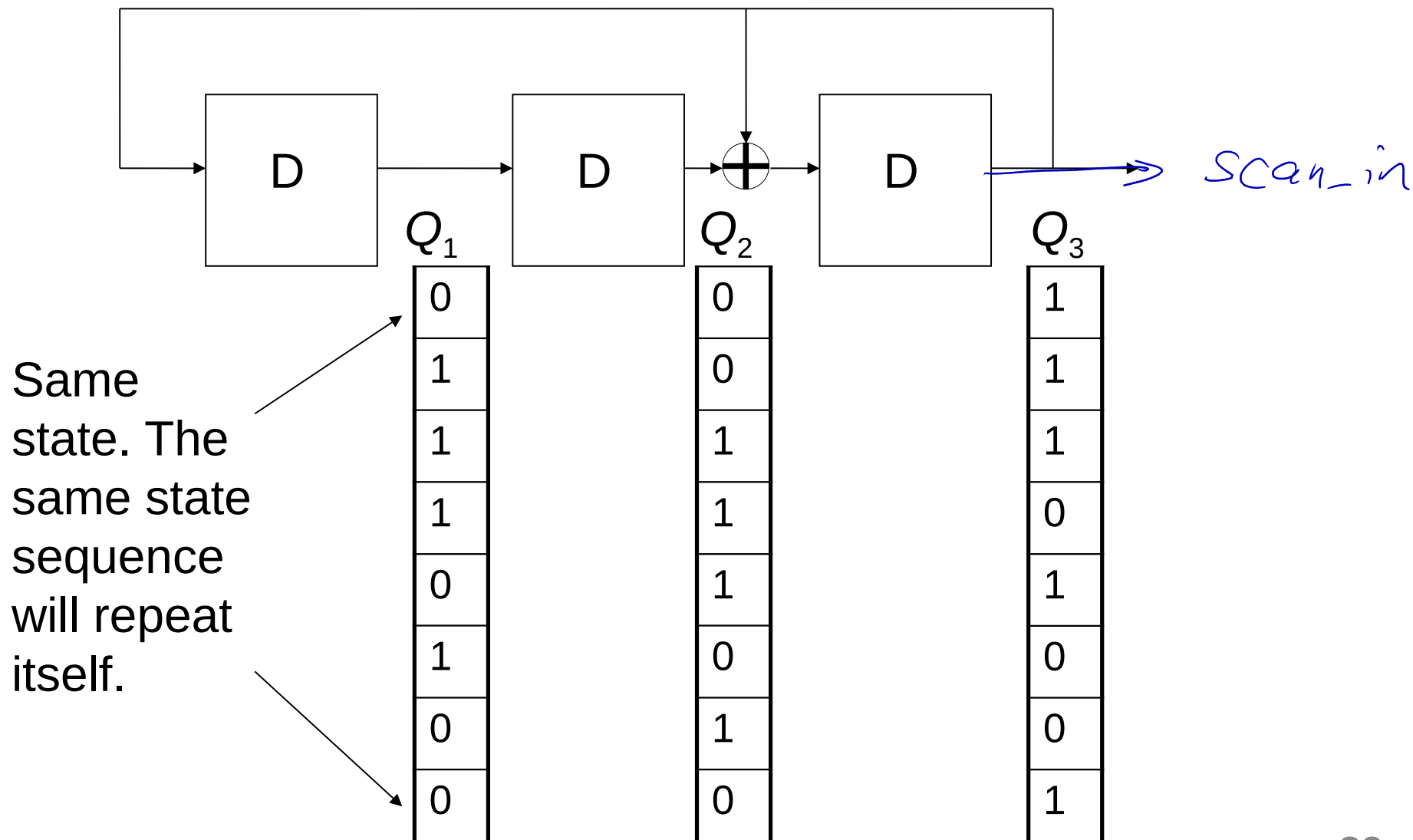


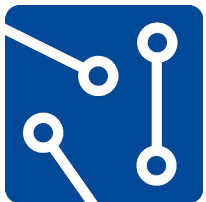
Example 1





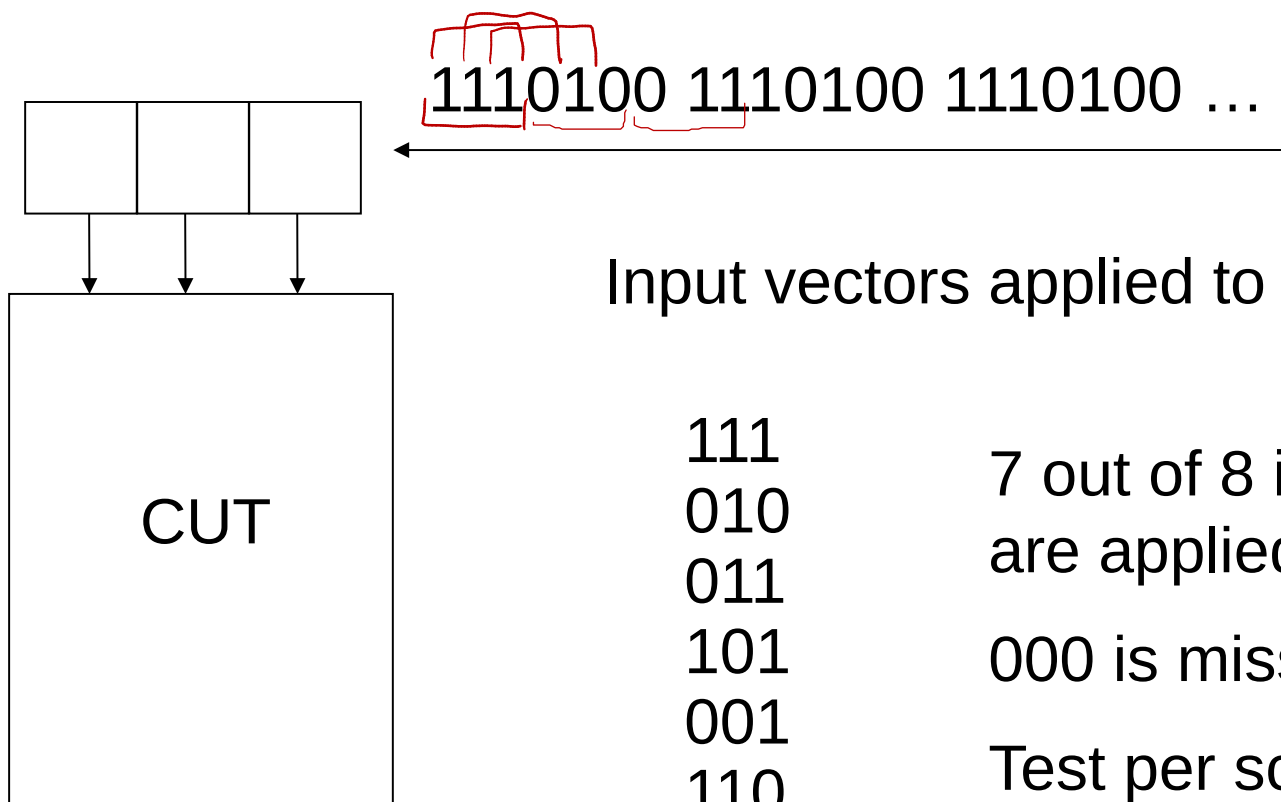
Example 1





→ output of the LFSR

Using Q3 to Drive a Scan Chain



Input vectors applied to the circuit:

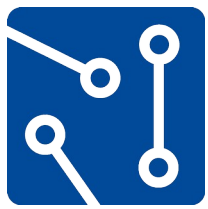
111
010
011
101
001
110
100

7 out of 8 input vectors
are applied to the CUT.

000 is missing.

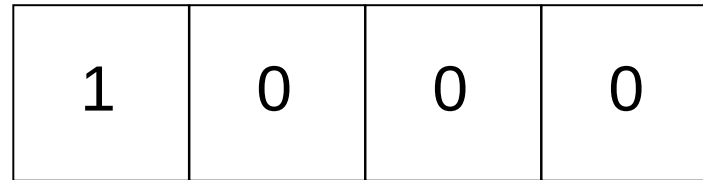
Test per scan. ←

Test per clock: 111, 110,
101, 010, ...



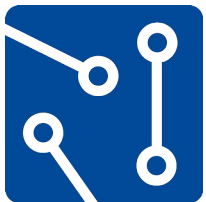
The State 0...0

- If an LFSR enters state 0...0, it will stay in this state.
- Consequently, this state is avoided during pattern generation.
- If 0...0 is necessary as a test pattern for the circuit-under-test, there are several methods to add it.
 - E.g. Use an LFSR which is longer by one bit.

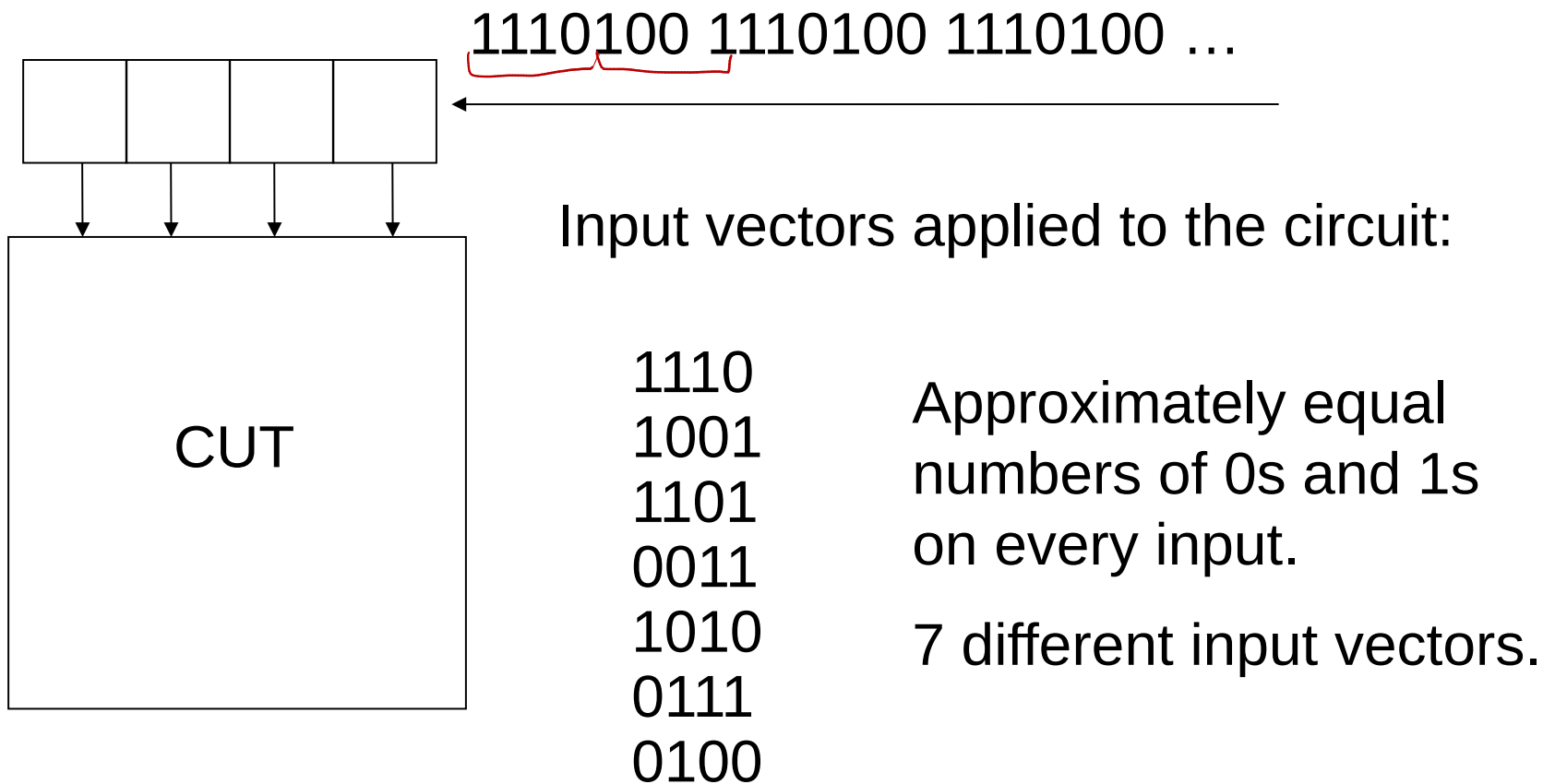


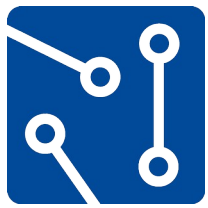
ensures that the LFSR will not stay in the required all-0 state

all-0 state

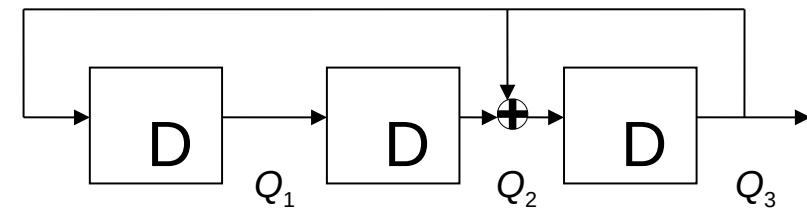


A Four-Input Circuit

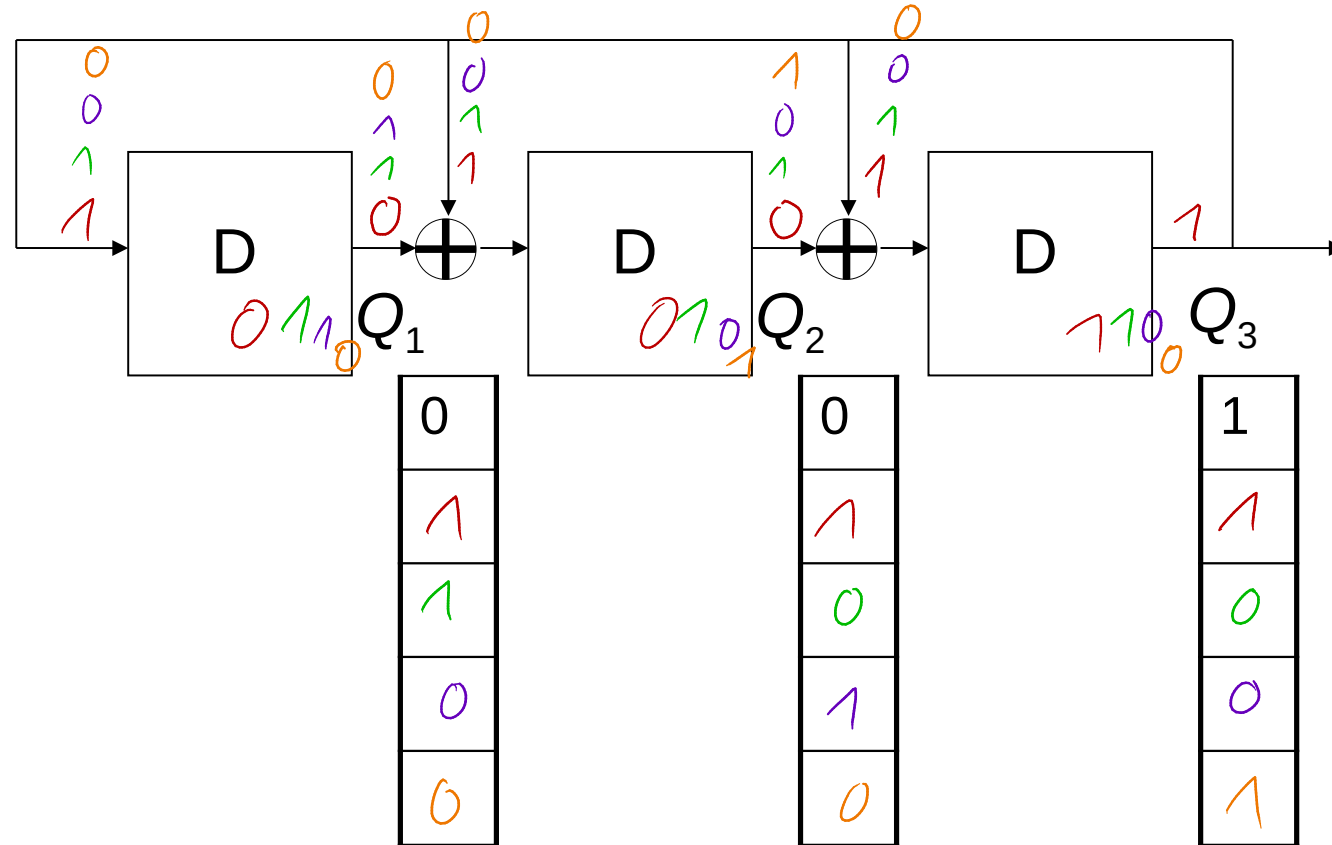


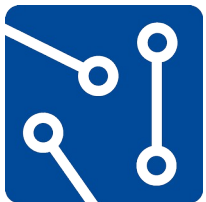


Example 2

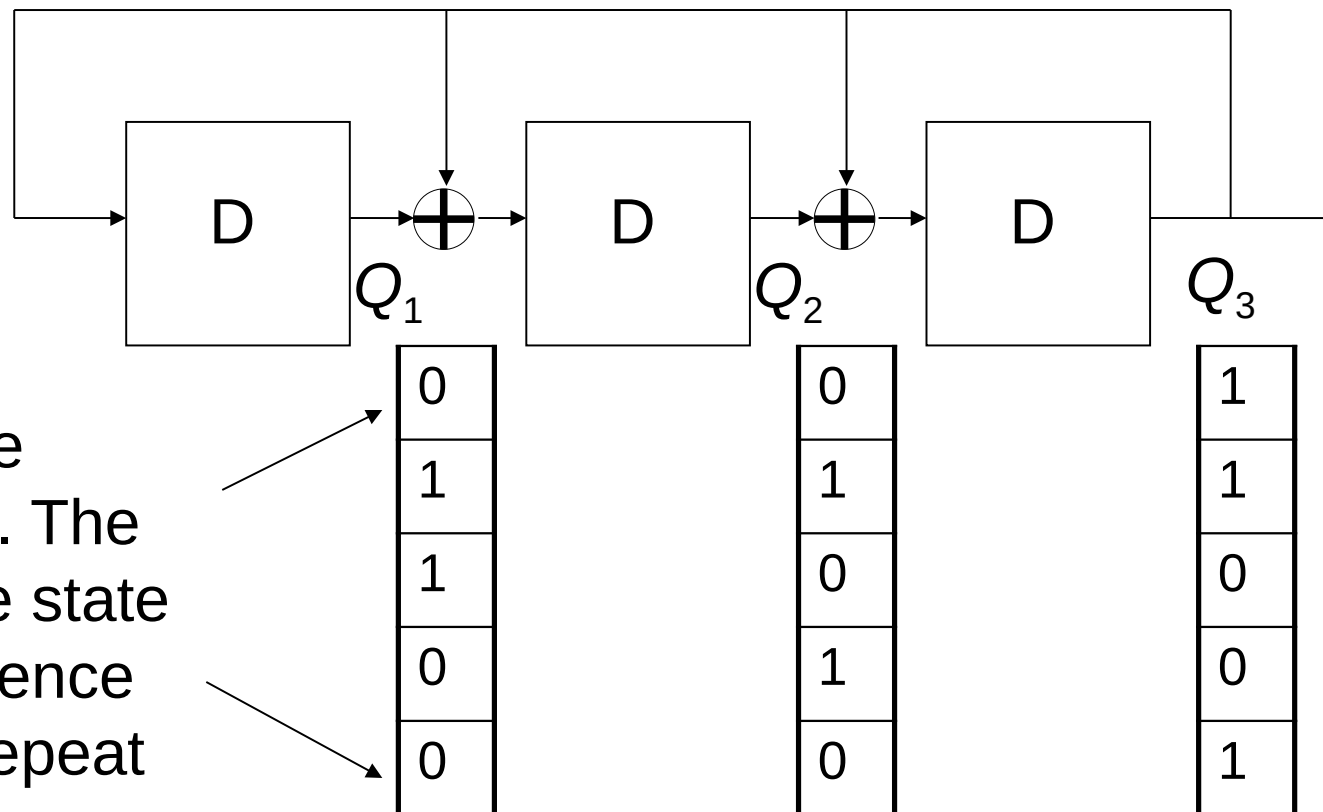
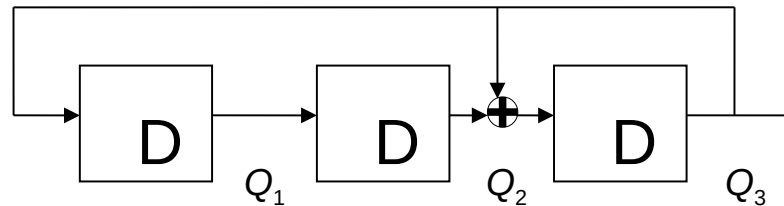


← LFSR
from
before

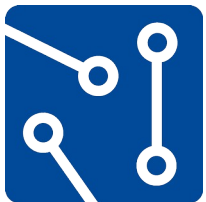




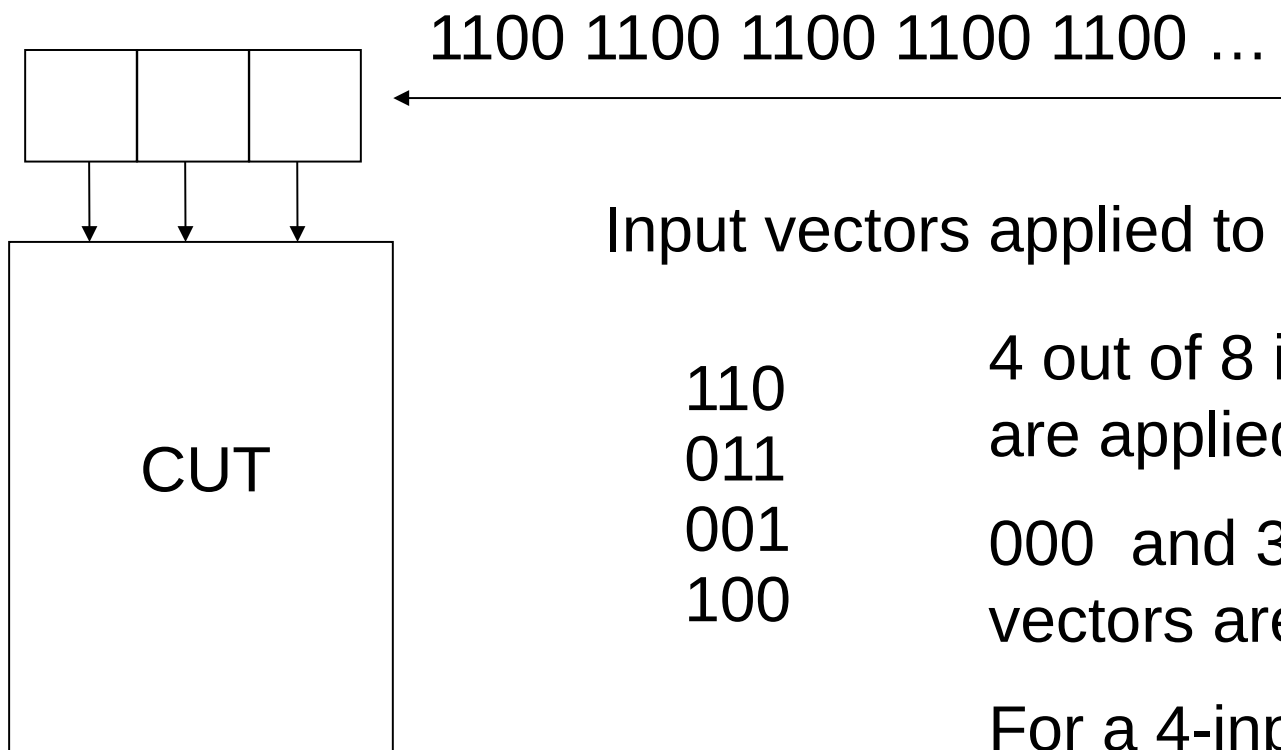
Example 2



Same state. The same state sequence will repeat itself.



Using Q3 to Drive a Scan Chain



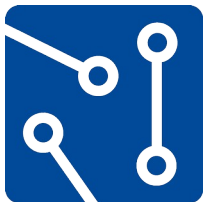
Input vectors applied to the circuit:

110
011
001
100

4 out of 8 input vectors
are applied to the CUT.

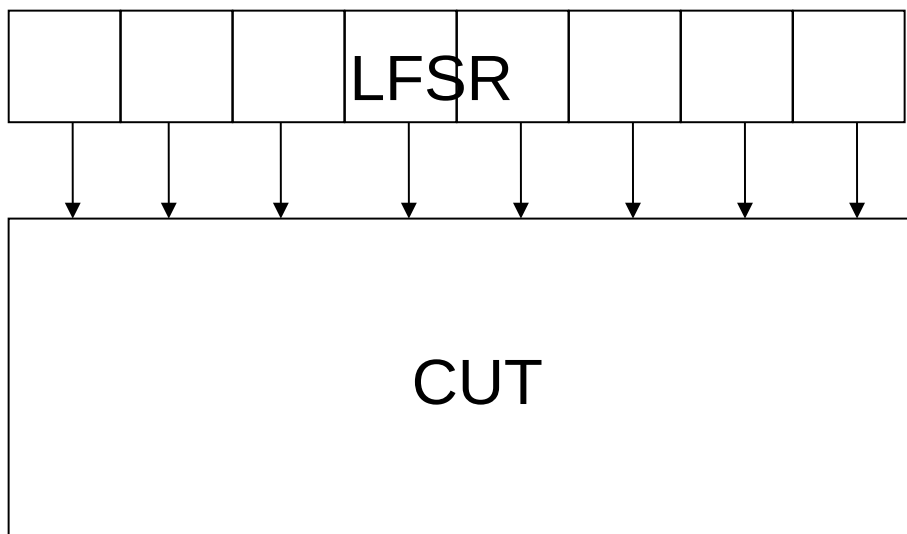
000 and 3 additional
vectors are missing.

For a 4-input circuit,
only one input vector
(1100) will be applied.

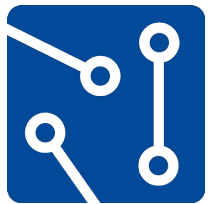


A Different Architecture

- It is possible to use the LFSR to drive the circuit inputs directly, not through a scan chain.

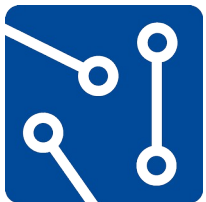


Expensive for
circuits with
large numbers
of inputs



Circuits with Large Numbers of Inputs

- The length of the LFSR should be large enough to **avoid a periodic set** of input vectors. For example, a 32-bit LFSR with a period of $2^{32}-1$.
- The length of the LFSR does **not have to be a function** of the number of circuit inputs.
- The LFSR will go through part of its cycle to apply input vectors to the circuit.



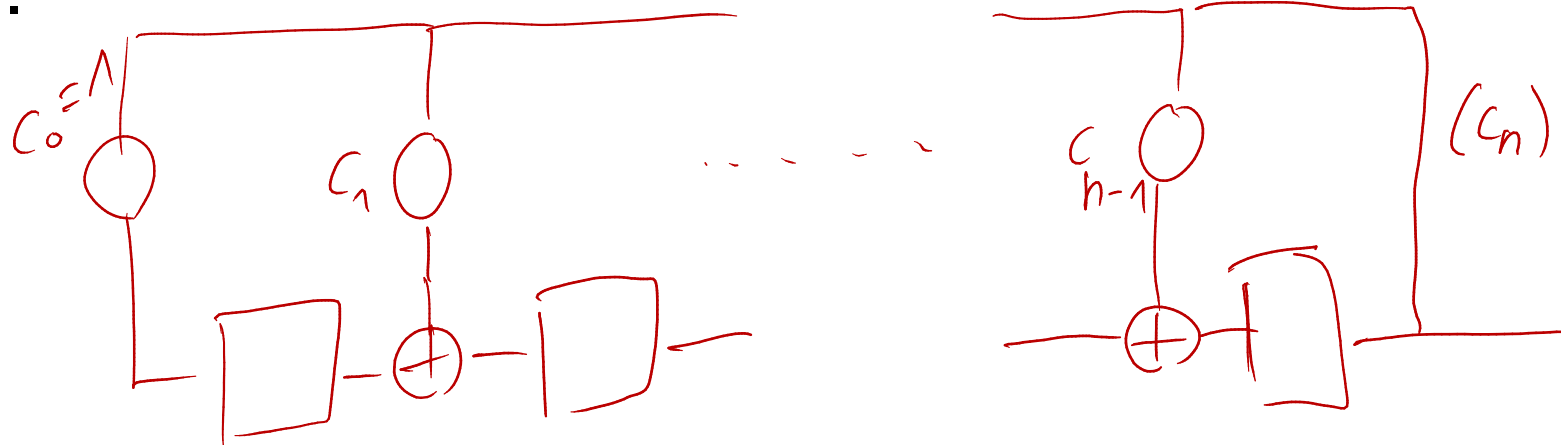
Characteristic Polynomials

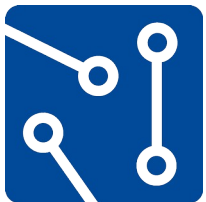
- An LFSR can be described by a polynomial in x (called the **characteristic polynomial**), where x indicates a delay by one time unit.

- For an LFSR with coefficients $c_1, c_2, \dots, c_n$, the polynomial is

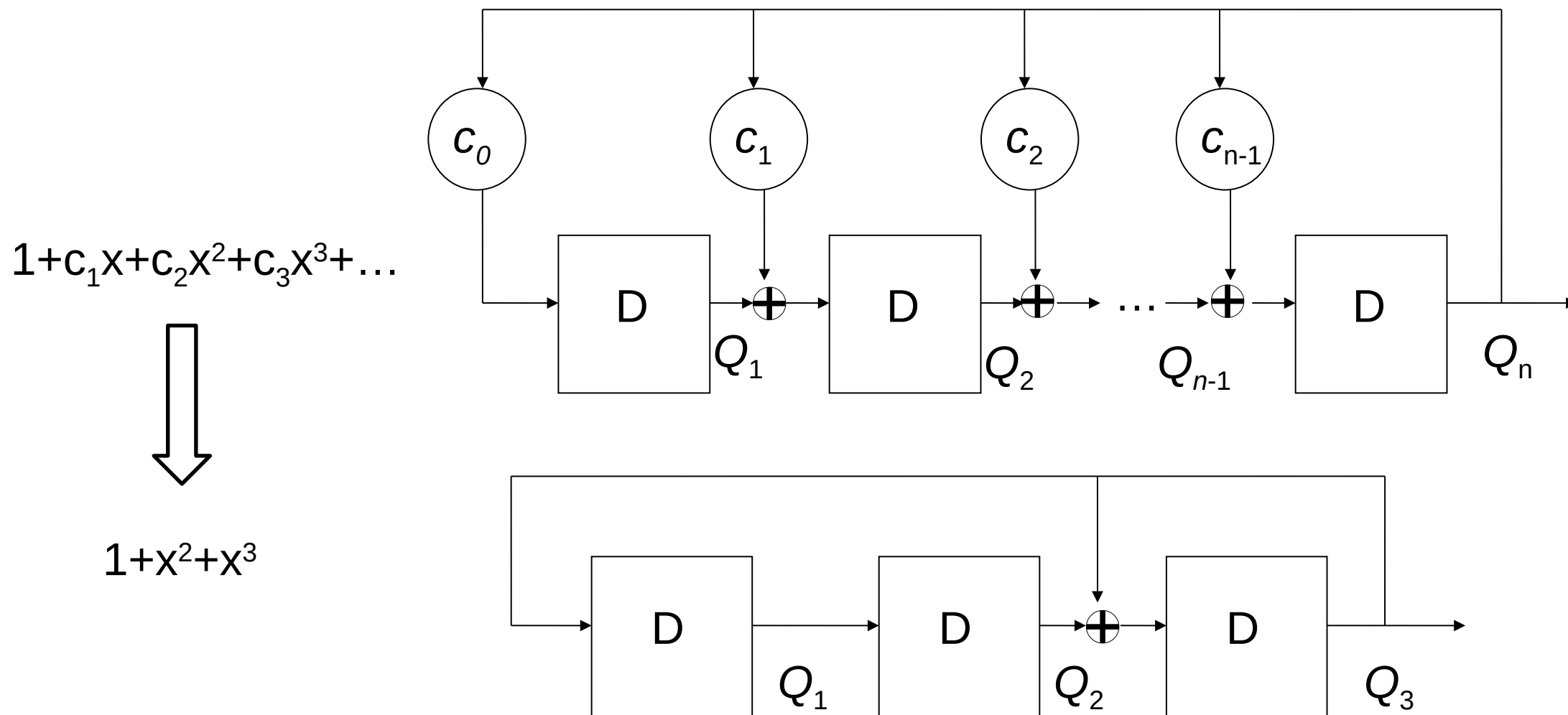
$$1 + c_1x + c_2x^2 + \dots + c_{n-1}x^{n-1} + c_nx^n$$

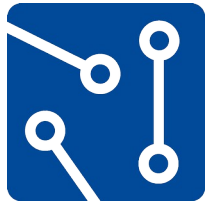
- The cycle length can be determined from properties of the polynomial.





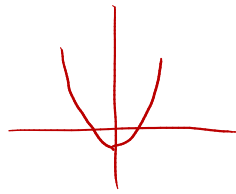
Characteristic Polynomial of Example 1





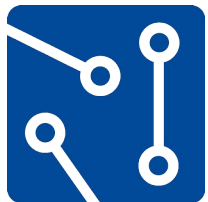
Properties of Polynomials

$$\begin{array}{l} X^2 - 1 = (x+1)(x-1) \\ X^2 + 1 \end{array}$$



- If the sequence generated by an n-stage LFSR has the period $2^n - 1$, it is called a **maximum length sequence**.
- The characteristic polynomial associated with a maximum-length sequence is called a **primitive polynomial**.
- An **irreducible polynomial** is one that cannot be factored, i.e., it is not divisible by any polynomial other than 1 and itself.
- A primitive polynomial is irreducible.
 - Find irreducible polynomials to get characteristic polynomials with a sequence of maximum length.

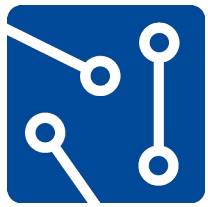
Def.: An irreducible polynomial P is primitive if the smallest possible integer k that allows the polynomial to divide into $1 + x^k$ occurs for $k = 2^n - 1$, where n is the degree of P .



Properties of Maximum Length Sequences

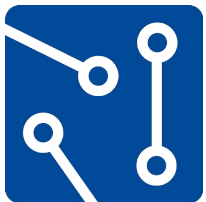
(00...0)
excluded

- The number of 1s is different from the number of 0s by one.
- The number of runs of 0s and 1s are equal.
- One half of the runs have length 1, one fourth have length 2, and so on.
- These properties are likely to **hold for a subset** of the maximum length sequence, making it useful for random vector generation.

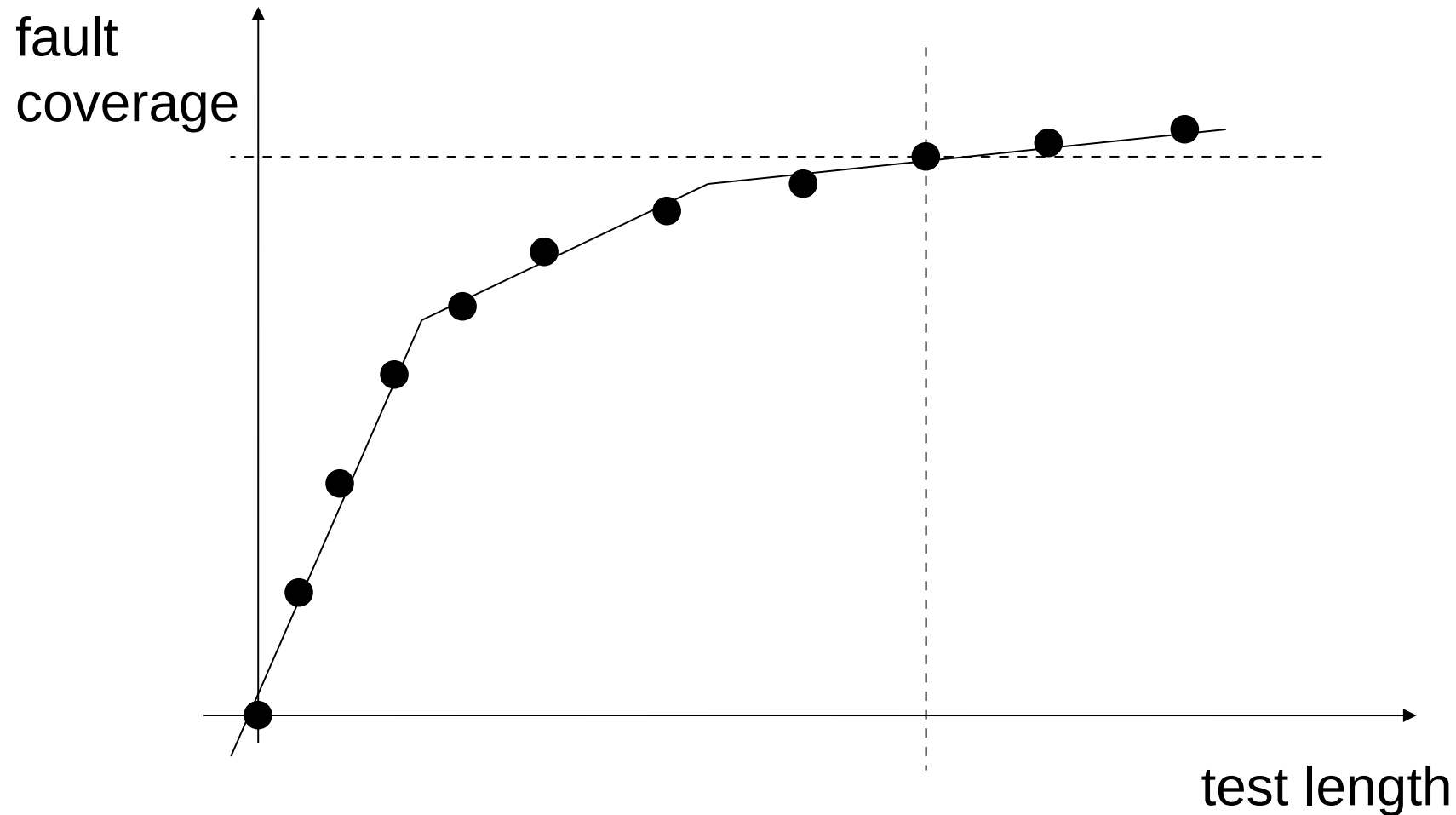


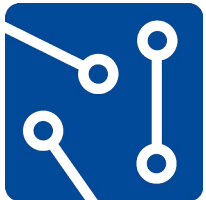
Determining the Fault Coverage and Test Length

- The fault coverage achieved by pseudo-random patterns produced by an LFSR is determined by **fault simulation**.
- The test length can be determined:
 - From fault detectability measures
 - Based on simulation results: apply random patterns until the fault coverage saturates.



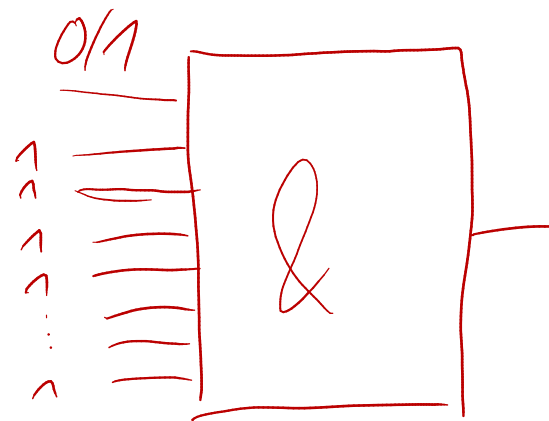
Fault Coverage vs. Test Length

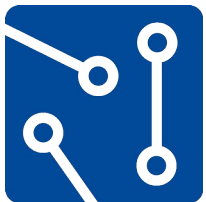




Weighted-Random Patterns (WRP)

- A fault may be difficult to **detect by random patterns**. Such a fault is called **random-pattern resistant**.
- A fault may be random-pattern resistant because:
 - The number of input vectors that detect it is small.
 - The input vectors that detect it have **significantly more** 1s than 0s (or more 0s than 1s).





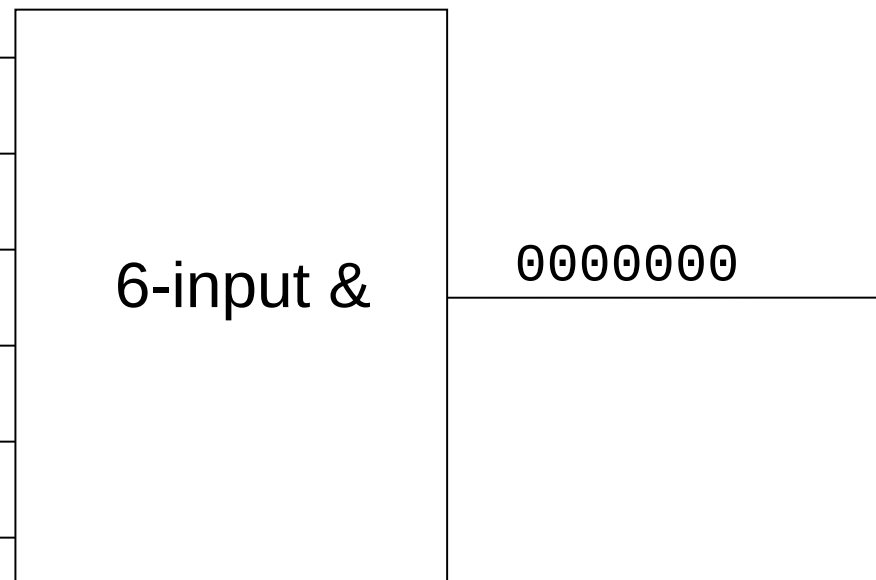
Example – Random Patterns

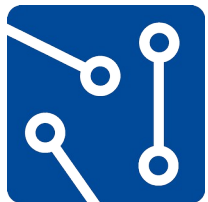
Typical random
patterns:

001111
010011
001111
100011
101100
010011
011000
111011
001001
000111

test patterns:

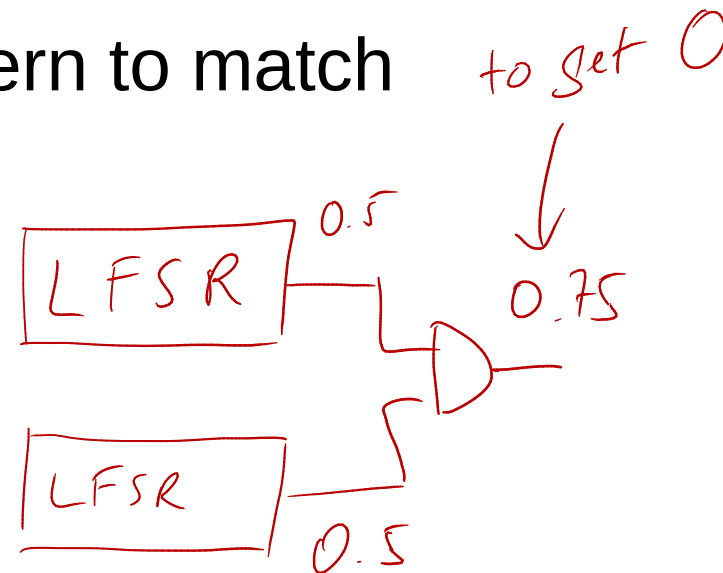
1011001
0101101
1000110
1101001
1111000
0010110

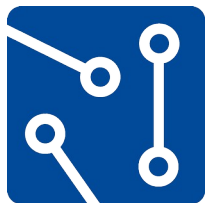




Possible Solution

- Bias the probability of 0s and 1s in a random pattern to match the input vectors required for testing the circuit.
- Called weighted random pattern generation.
- It is possible to use:
 - The same weight for all the inputs.
 - Different weights for different inputs.
 - Multiple sets of weights for different random pattern resistant faults.





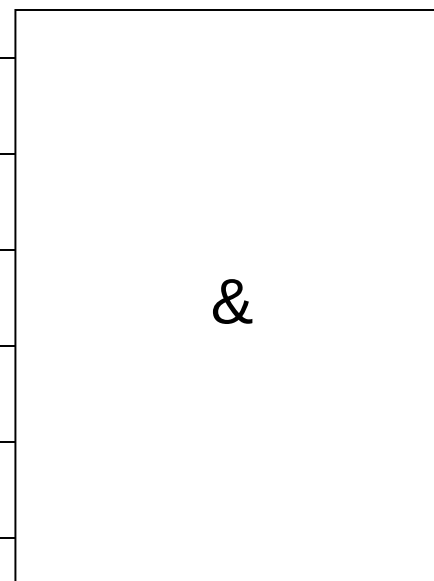
Example – Weighted-Random Patterns with Weight 3/4

Random
patterns:

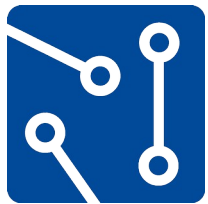
111111
011111
101101
111110
011101
011011
110111
111111
111011
110101

test patterns:

1111001
0101111
1110011
0111011
1111101
1111111

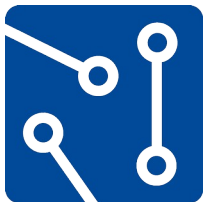


01000001

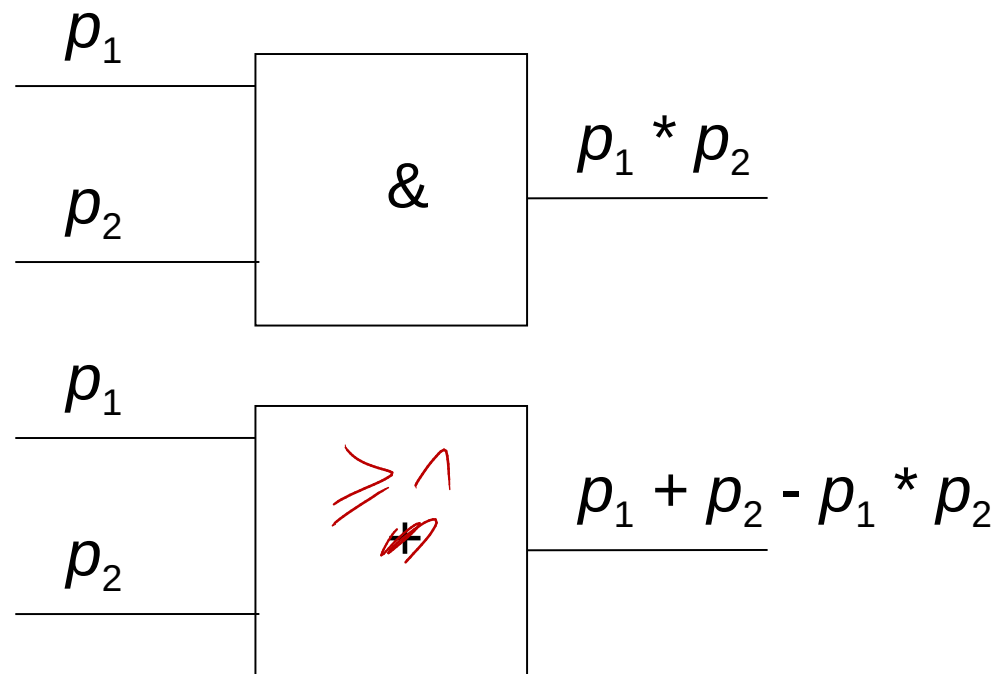


Implementing Weighted-Random Patterns

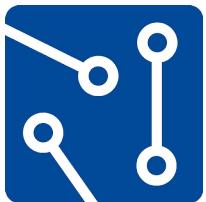
- An LFSR produces a 1 output with probability 0.5. An LFSR cell output is 1 with probability 0.5.
- The **AND** function of two LFSR cell outputs is 1 with probability $\frac{1}{4}$.
- The **OR** function of two LFSR cell outputs is 1 with probability $\frac{3}{4}$.
- To obtain probability $\frac{1}{2^i}$ use an AND gate connected to i LFSR cells.
- To obtain probability $1 - \frac{1}{2^i}$ use an OR gate connected to i LFSR cells.



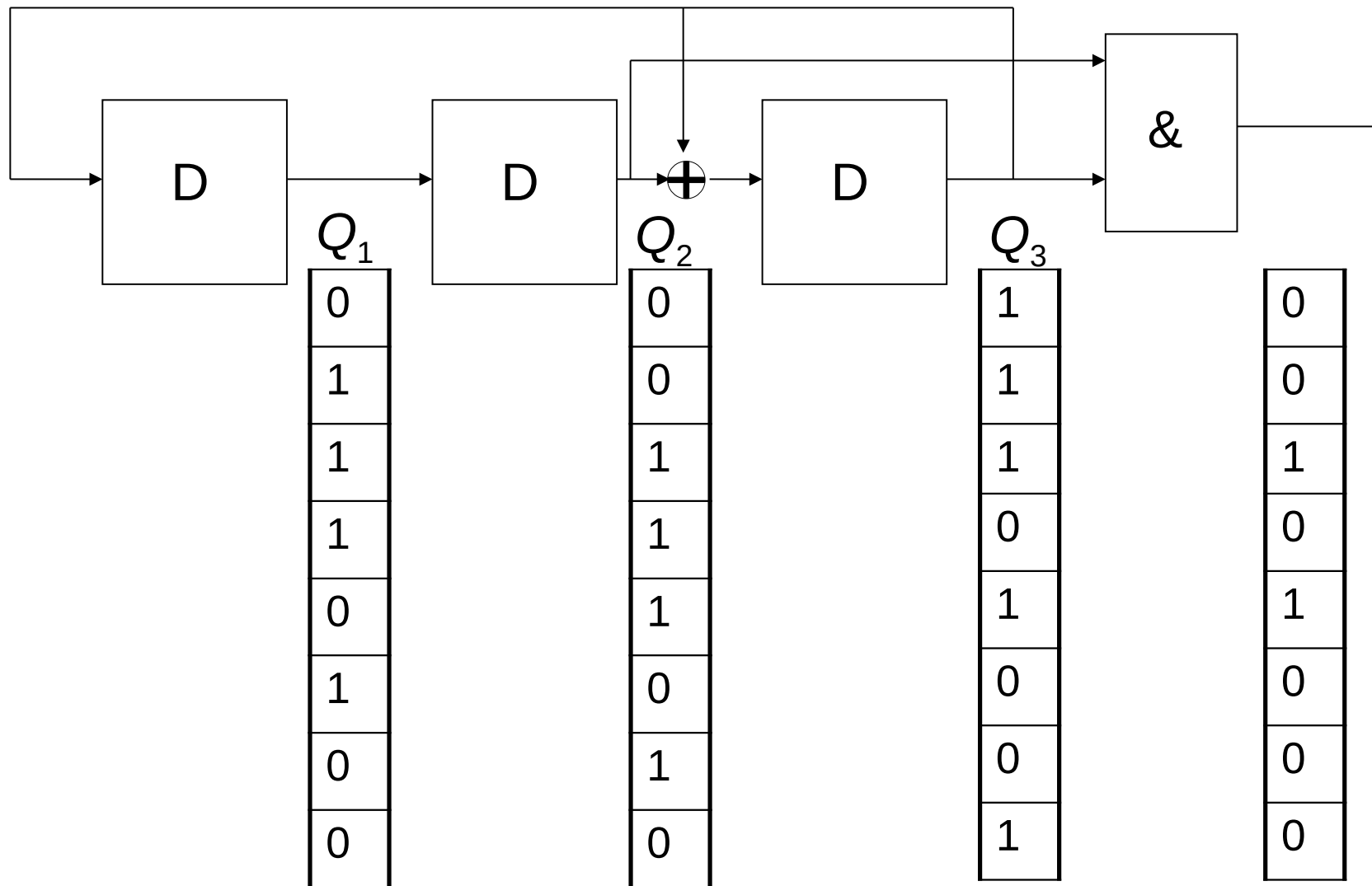
Dependent Probabilities



Assumption: p_1 and p_2 are independent.
Not accurate for an LFSR.

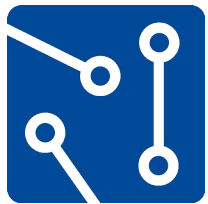


Example



biased

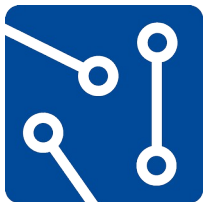
biased



Selecting Weights for a Circuit

- Based on structural analysis of the circuit (**fault detectability analysis**).
- Based on a deterministic test set (**with rounding**).

0	0	1	1
0	0	0	0
0	0	0	1
0	0	1	1
0	1	0	1
$p = 0$	$p = 0.2$ $\rightarrow 0.25$	$p = 0.4$ $\rightarrow 0.5$	$p = 0.8$ $\rightarrow 0.75$



The Need for Multiple Sets of Weights

Test patterns:

11111

01111

10111

11011

11101

11110

00000

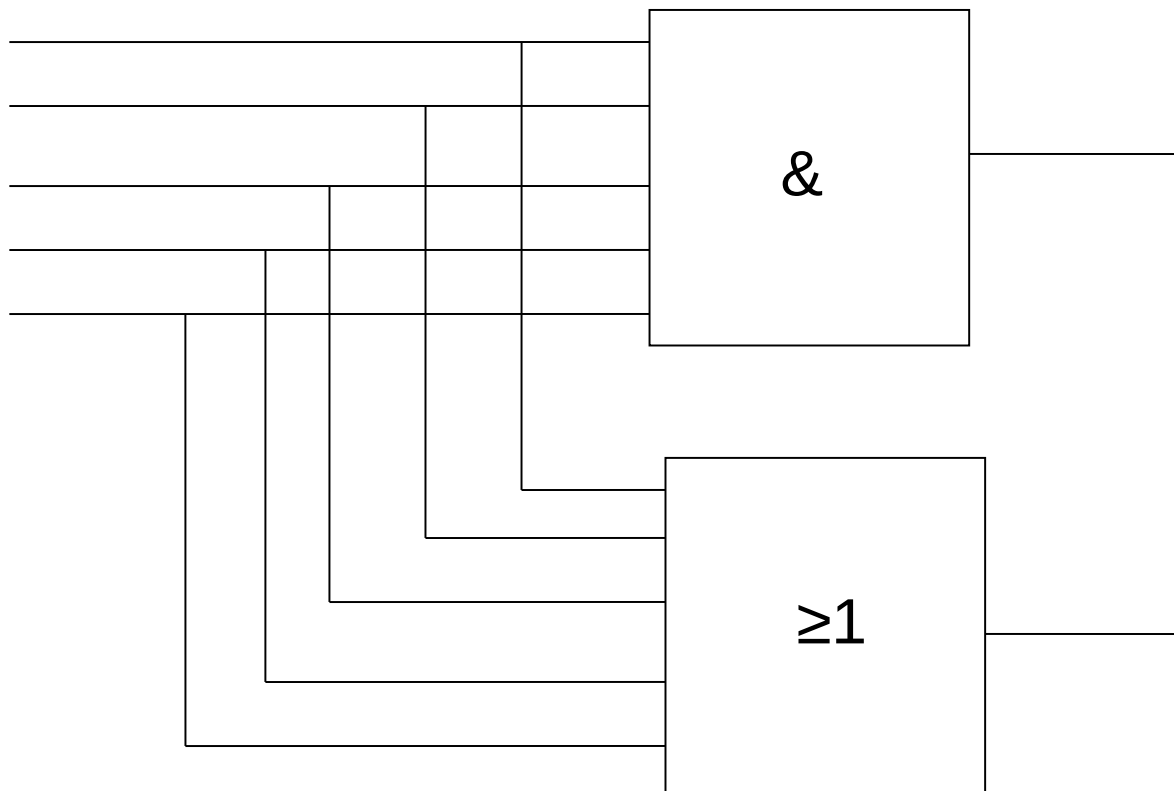
10000

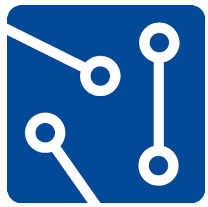
01000

00100

00010

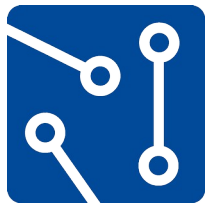
00001





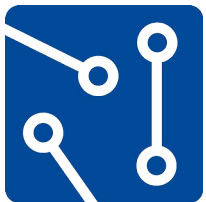
Multiple Sets of Weights

- Select a set of weights based on a **deterministic test set** or a subset of it.
- Apply a **preselected number of weighted random tests** and find the set of detected faults.
- **Drop deterministic patterns** that do not detect any undetected faults.
- Repeat until all the faults are detected.



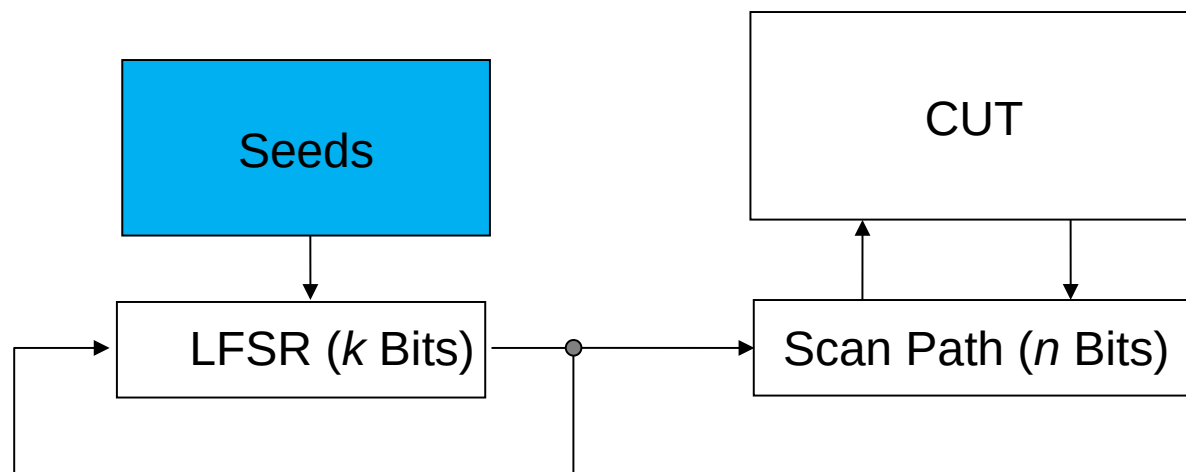
Advanced Techniques for High Quality Non-Intrusive BIST

- Deterministic and mixed mode BIST
 - For circuits requiring **many different weight distributions**, the overhead for WRP may become unaffordable.
 - Mixed mode scan-based schemes efficiently **combine** the advantages of **pseudo-random and deterministic BIST**.
- Several successful approaches:
 - Store and generate, e.g. **reseeding of LFSRs**
 - Predict and correct, e.g. **bit-fixing/bit-flipping**

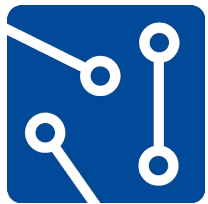


Reseeding of LFSRs (1)

Koenemann 1991:

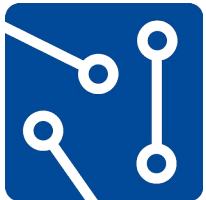


- Reprogrammable and flexible hardware

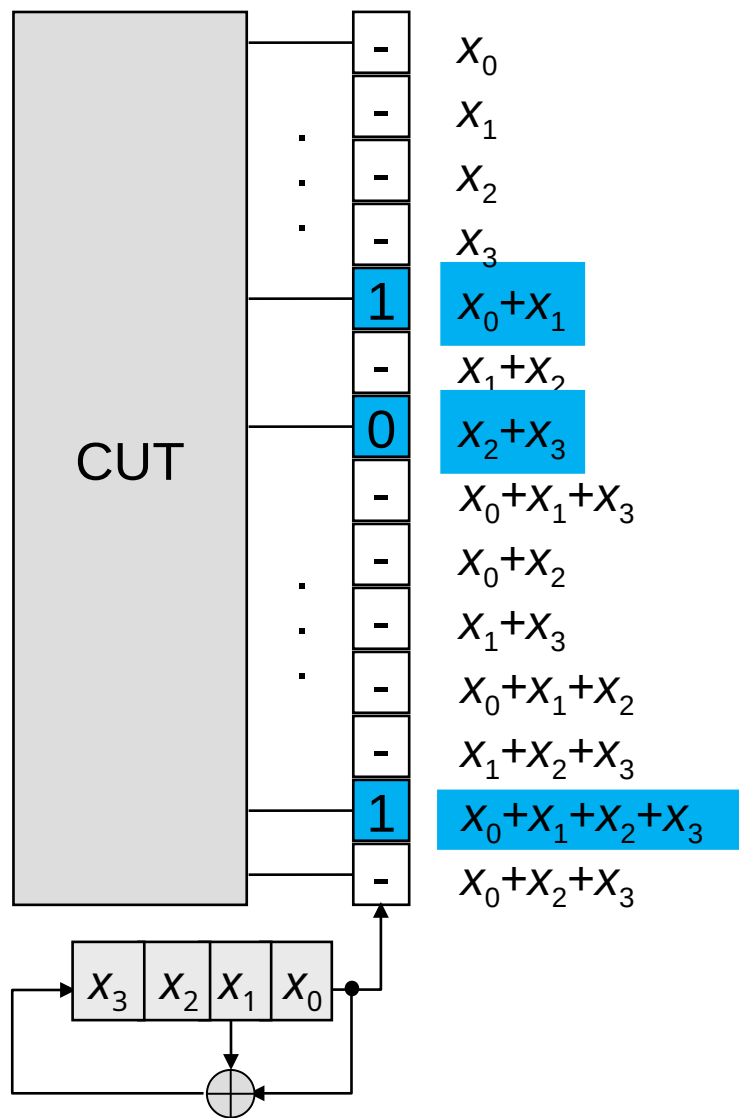


Reseeding of LFSRs (2)

- Simple **pseudo-random** test catches **most of the faults**.
- A **few deterministic** patterns detect the **remaining hard faults**.
- Storing the deterministic patterns would require n bits (length of the scan chain) per pattern.
- Encoding the patterns as seeds of the LFSR only requires k bits per pattern (“store and generate”).
- Encoding is simple: linear equation for each pattern.



Reseeding of LFSRs (3) - Encoding



Test cube $t \in \{0,1,-\}^{14}$:

$(-, -, -, -, 1, -, 0, -, -, -, -, 1, -)$

Equations:

$$x_0 + x_1 = 1$$

$$x_2 + x_3 = 0$$

$$x_0 + x_1 + x_2 + x_3 = 1$$

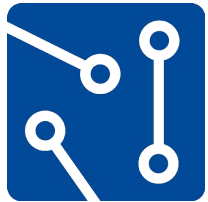
Solutions:

$$x_0 = x_2 = x_3 = 1$$

$$x_1 = 0$$

Shift register sequence:

$(1, 0, 1, 1, 1, 1, 0, 0, 0, 1, 0, 0, 1, 1)$



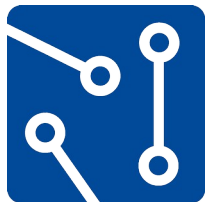
Reseeding of LFSRs (4) - Encoding

- Probability of successful encoding
probability of linear independent equations

For s specified bits and k -bit LFSR:

$$p(\text{independent}) = \prod_{i=0}^{s-1} \frac{2^k - 2^i}{2^k - i - 1}$$

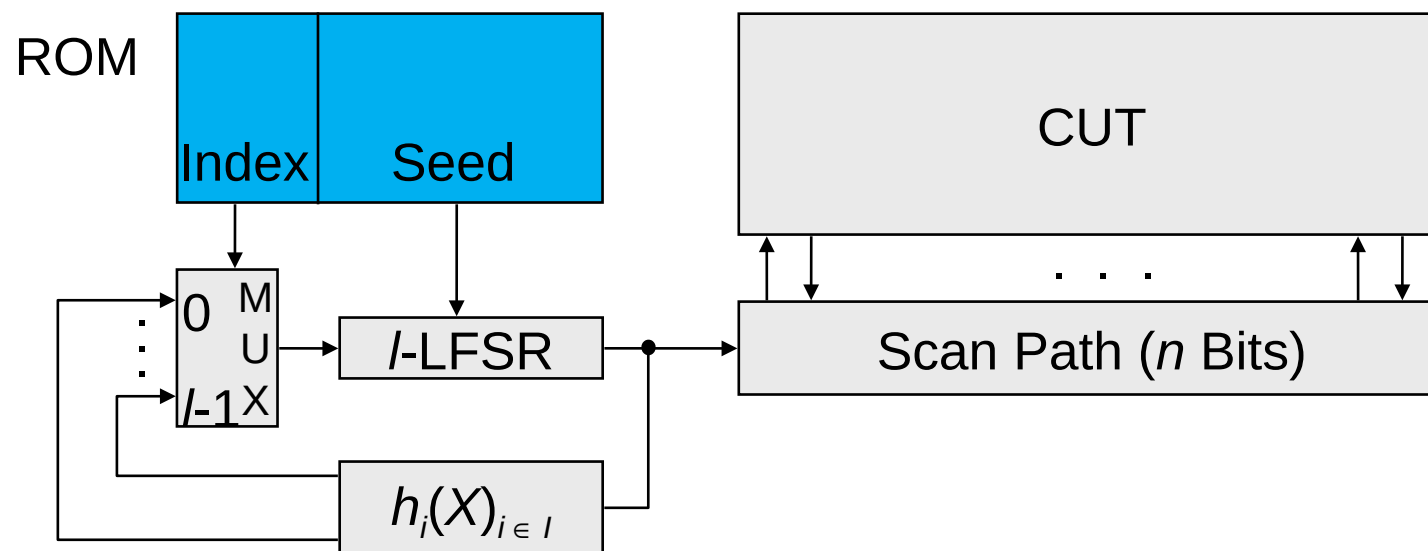
- To guarantee encoding with probability of $1 - 10^{-6}$:
 $k = s + 20$ is sufficient.
- The number s of specified bits is **limited in real life circuits**.

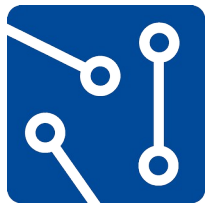


Reseeding of LFSRs (5) - Multiple Polynomial LFSRs

Hellebrand et al. 1992, 1995:

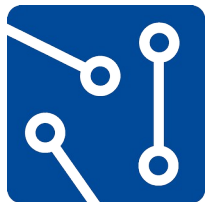
An I -LFSR implements a set $h_i(X)_{i \in I}$ of feedback polynomials.





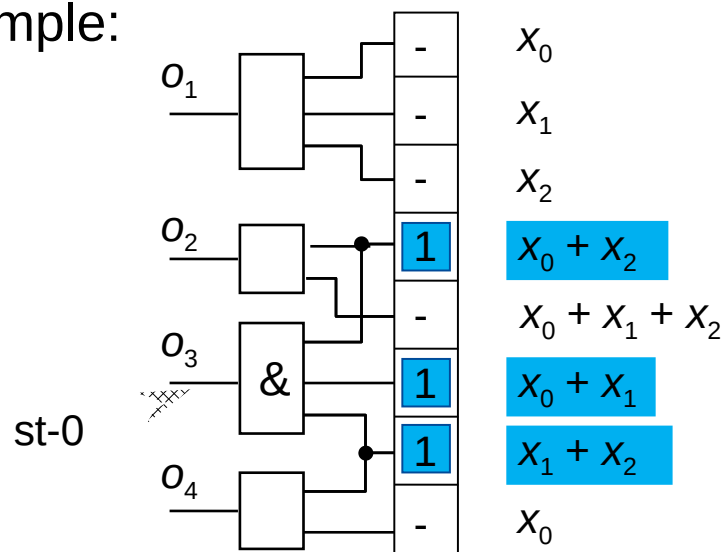
Reseeding of LFSRs (6) - Multiple Polynomial LFSRs

- Patterns which cannot be **encoded by one** LFSRs may be encoded **by another** LFSR.
- A **few shorter** LFSRs provide the **same quality as one long** LFSR.
- Only the **different feedback functions** have to be implemented.

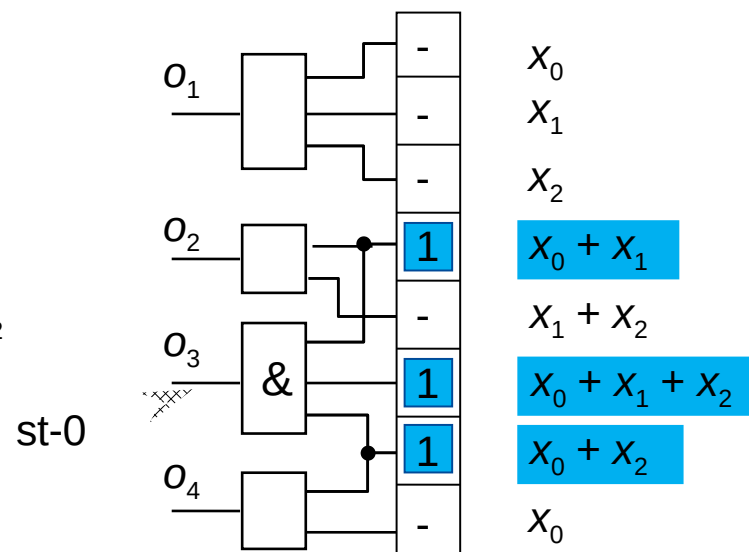


Reseeding of LFSRs (7) - Multiple Polynomial LFSRs

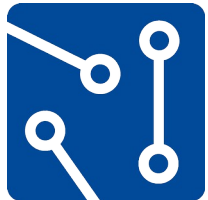
Example:



no solution!

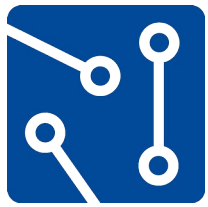


solution (0,0,1)



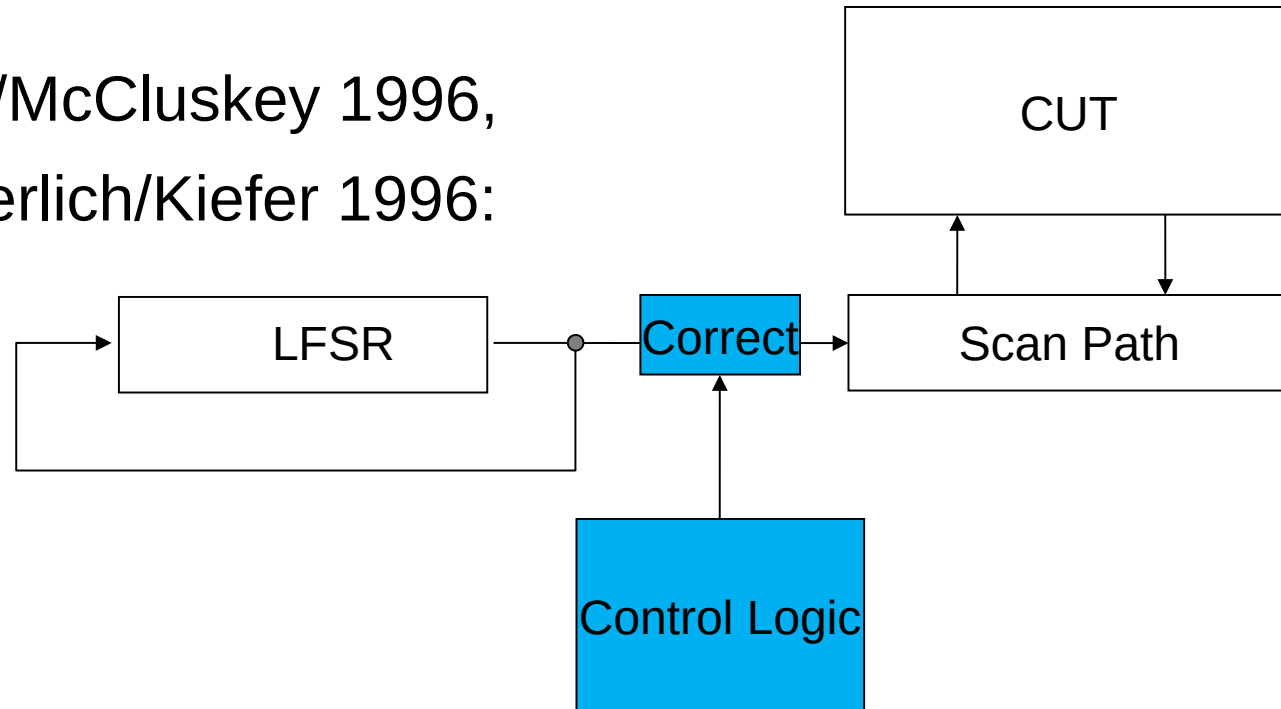
Reseeding of LFSRs (8) - Multiple Polynomial LFSRs

- Analysis:
 - For s specified bits 16 LFSRs of length s provide the same quality as one LFSR of length $s + 20$
 - The number of bits to be stored for one deterministic pattern reduces from $s + 20$ to $s + 4$
- Further optimization potential: s **varies** within a test set
 - Merging of patterns
 - Concatenation of patterns
 - Variable length seeds
 - Tailored ATPG

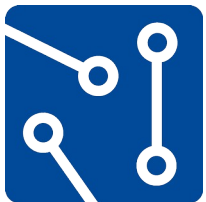


Bit Fixing/Bit Flipping (1)

Touba/McCluskey 1996,
Wunderlich/Kiefer 1996:

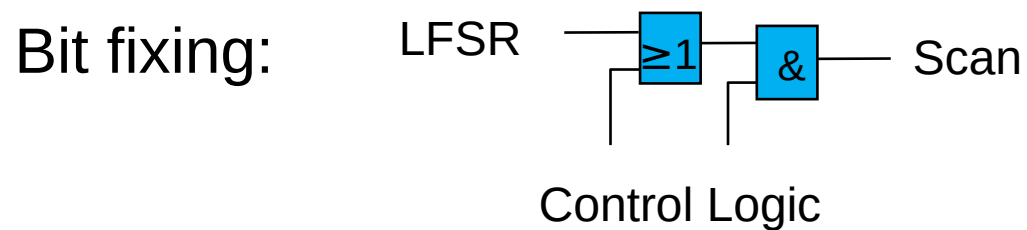


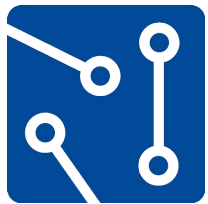
- Very efficient solution, cf. Kiefer et. al. ITC 2000
- Hardware tailored to a specific circuit



Bit Fixing/Bit Flipping (2)

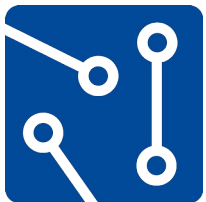
Correction of LFSR patterns:



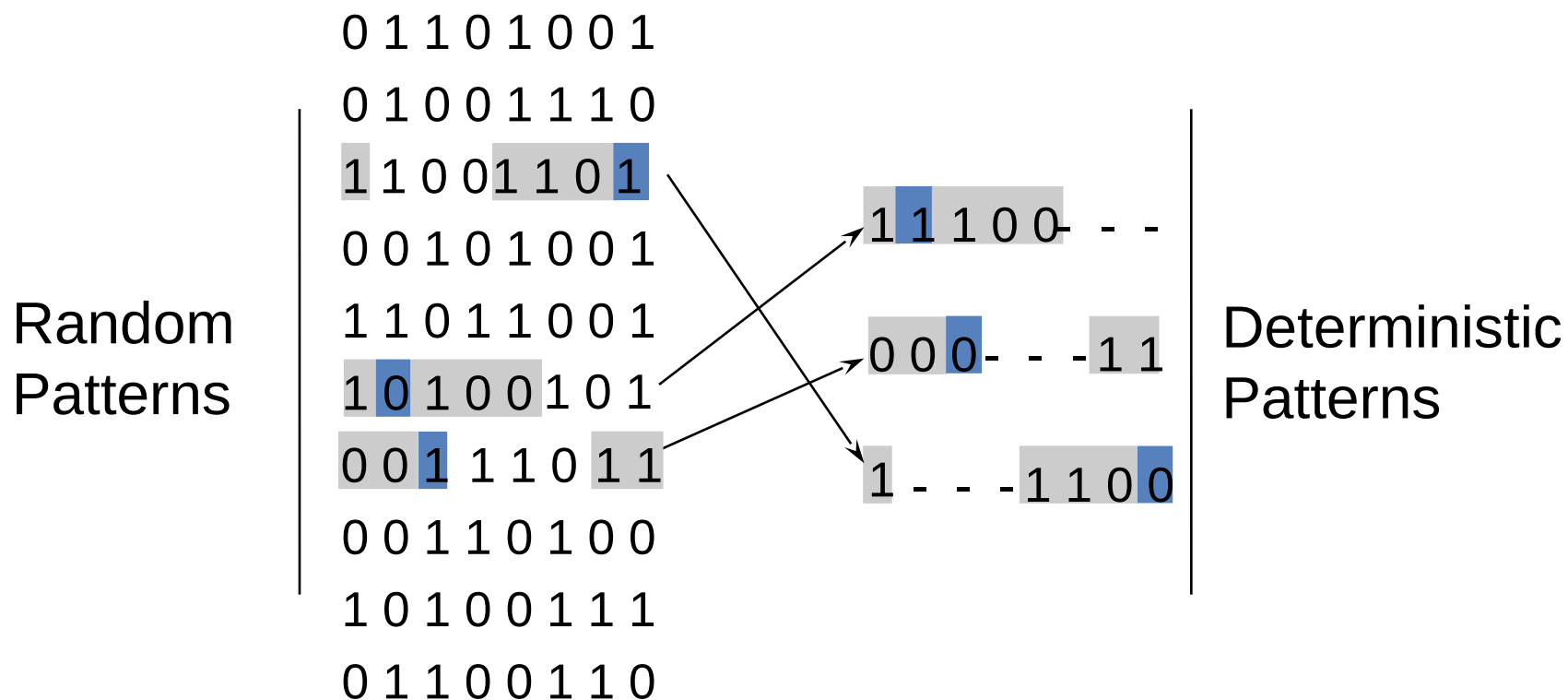


Bit Fixing/Bit Flipping (3)

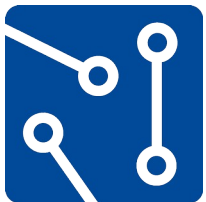
- The LFSR patterns detect most of the faults, but **many of the patterns are “useless”** (do not detect any new faults).
- The **remaining hard faults** can be detected by a few deterministic patterns.
- Useless LFSR patterns can be **corrected to deterministic patterns** at low cost (“predict and correct”).
- Deterministic patterns are embedded into the LFSR sequence.



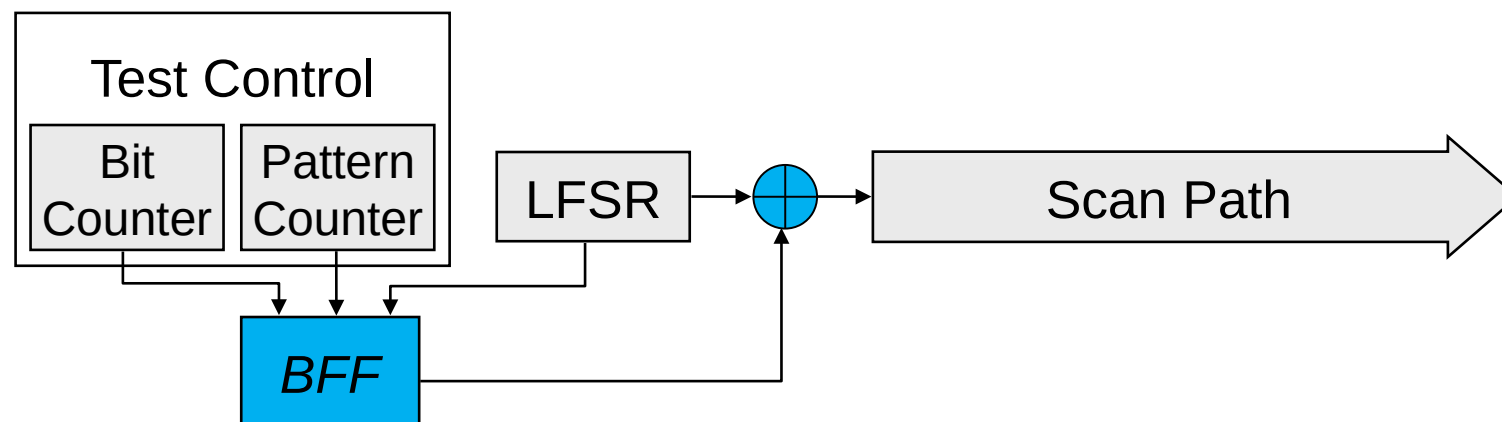
Bit Flipping (1) - Example



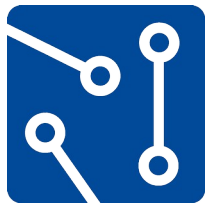
Example: 10,000 patterns, pattern with 20 specified bits
→ expect **1.79** bits per pattern to be flipped



Bit Flipping (2) - Target Structure

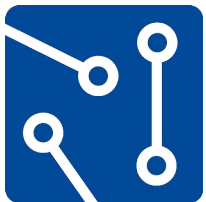


BFF = Bit-Flipping Function

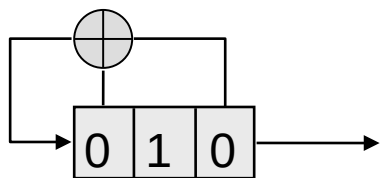


Bit Flipping (3) - LFSR Size

- Standard approach:
 - LFSR size $\log_2$ (test patterns)
 - e.g.: 10,000 patterns → LFSR size 14
 - Avoid linear dependencies for small LFSRs
 - e.g.: 32-bit data path → LFSR size 32
- Using BIST control:
 - LFSR size $< \log_2$ (test patterns)



Bit Flipping (4) - TPG Example



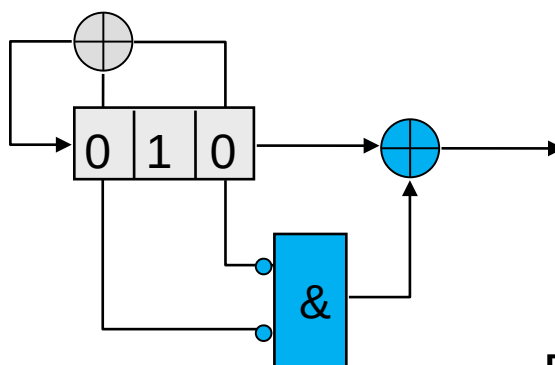
0 1 0 0 1

1 1 0 1 0

0 1 1 1 0

1 0 0 1 1

1 0 1 0 0



1 1 0 0 1

1 1 1 1 0

0 1 1 1 1

1 0 0 1 1

1 1 1 0 0

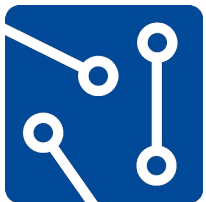
Fixed patterns:

1 1 - - -

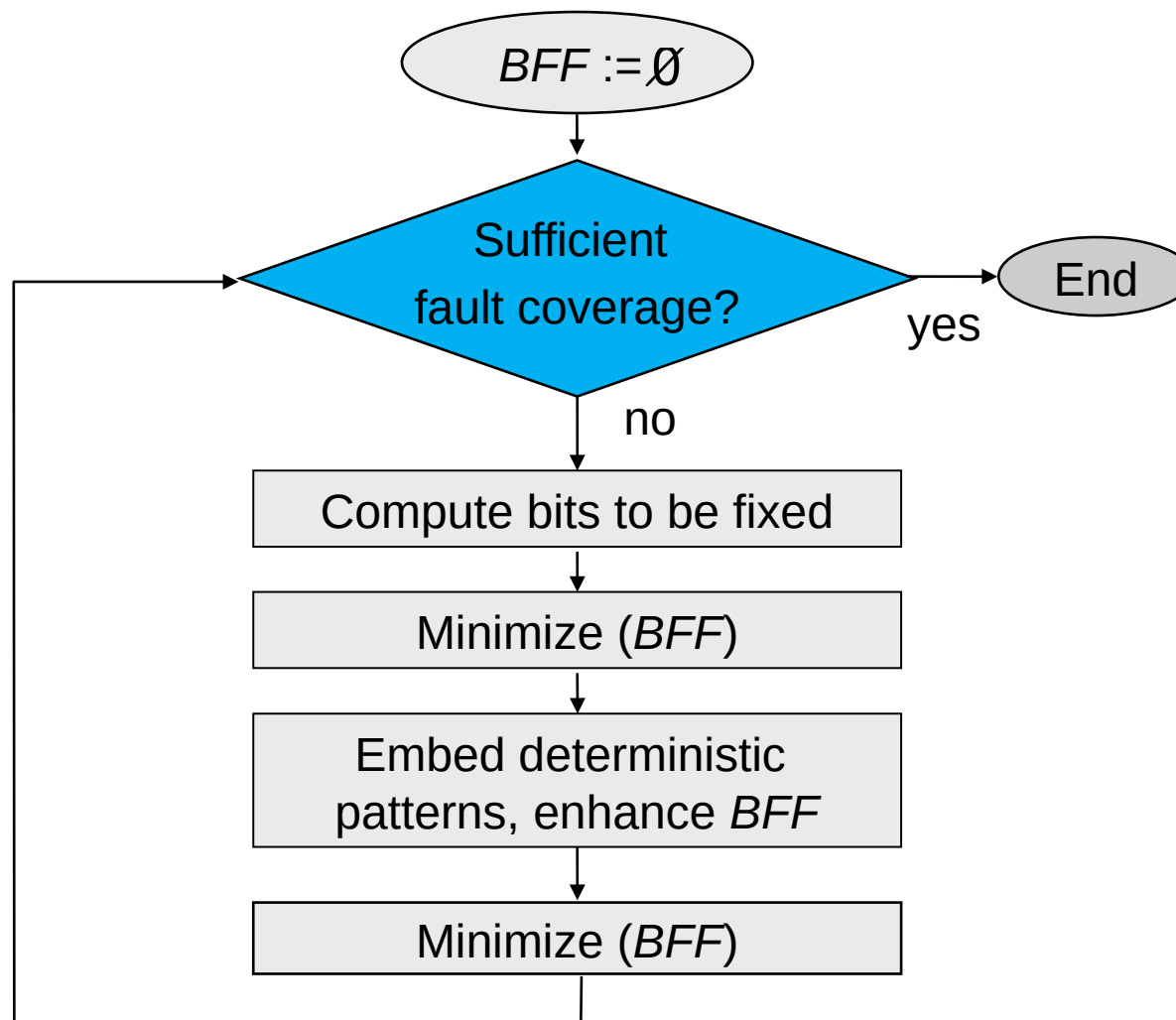
0 - - 1 -

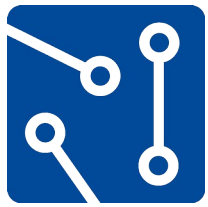
Mapped pattern:

1 1 - 0 1



Bit Flipping (5) - Synthesis of TPG

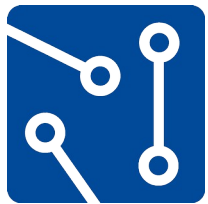




Industrial Experience

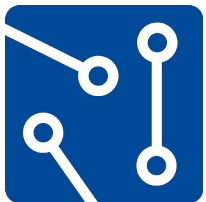
- Evaluation at Philips (IEEE ETW'00)

- 22 hard-to-test modules extracted from:
 - Programmable DSP core (~57K gates)
 - Graphics controller (~350K gates)
 - 80c51 microcontroller (~20K gates)
 - Telecom chip (~27K gates)
 - ...

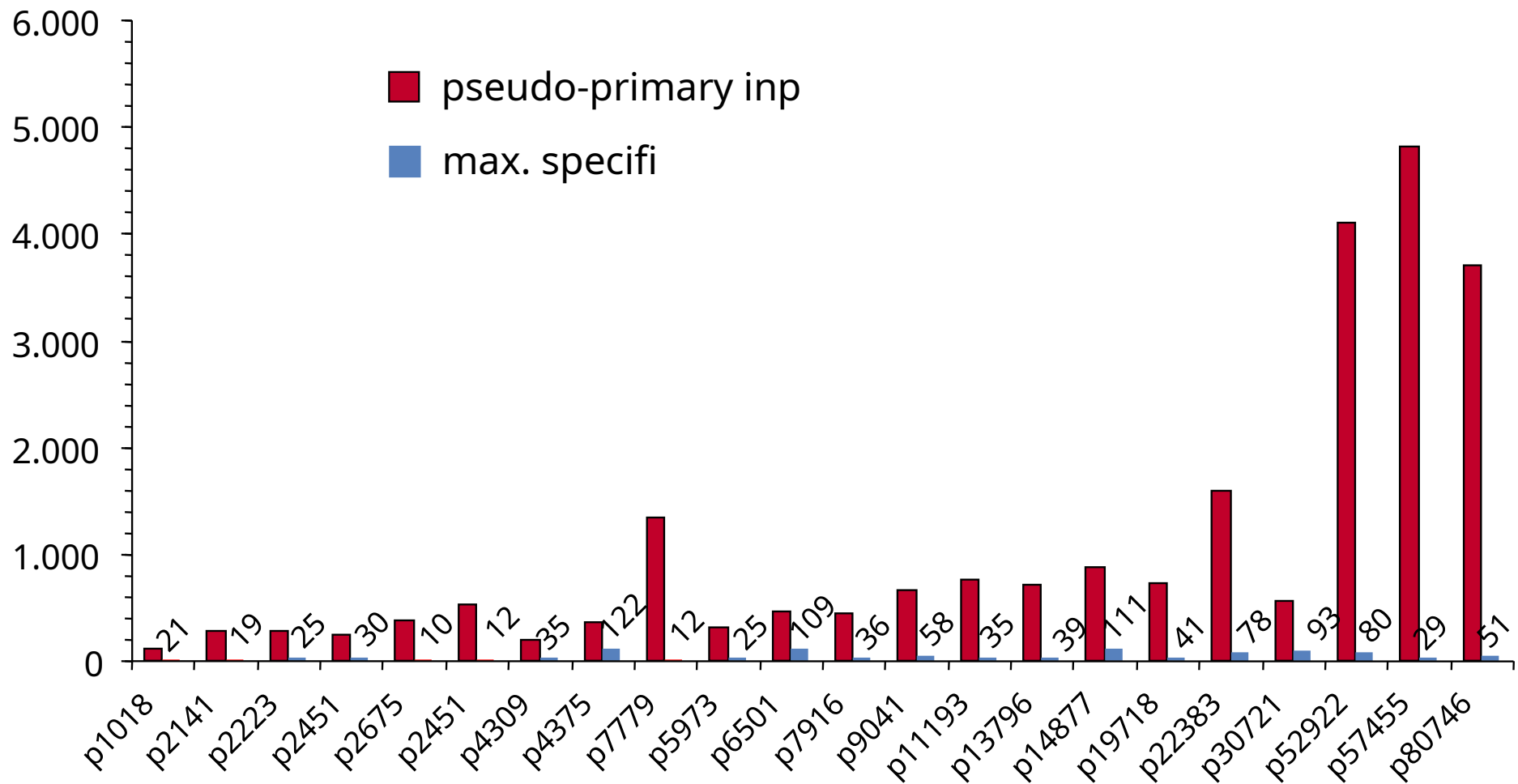


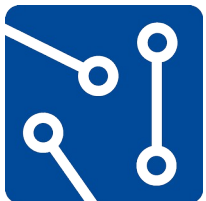
Experimental Setup

- Test length
 - 10,000 patterns → pessimistic!
- Target fault coverage
 - 100% of all detectable faults
- Varying parameters
 - LFSR size
 - LFSR polynomial
- Area estimation
 - Philips 0.25 μm CMOS technology
 - 1 μm CMOS technology, BFF: PLA macro cell(s)



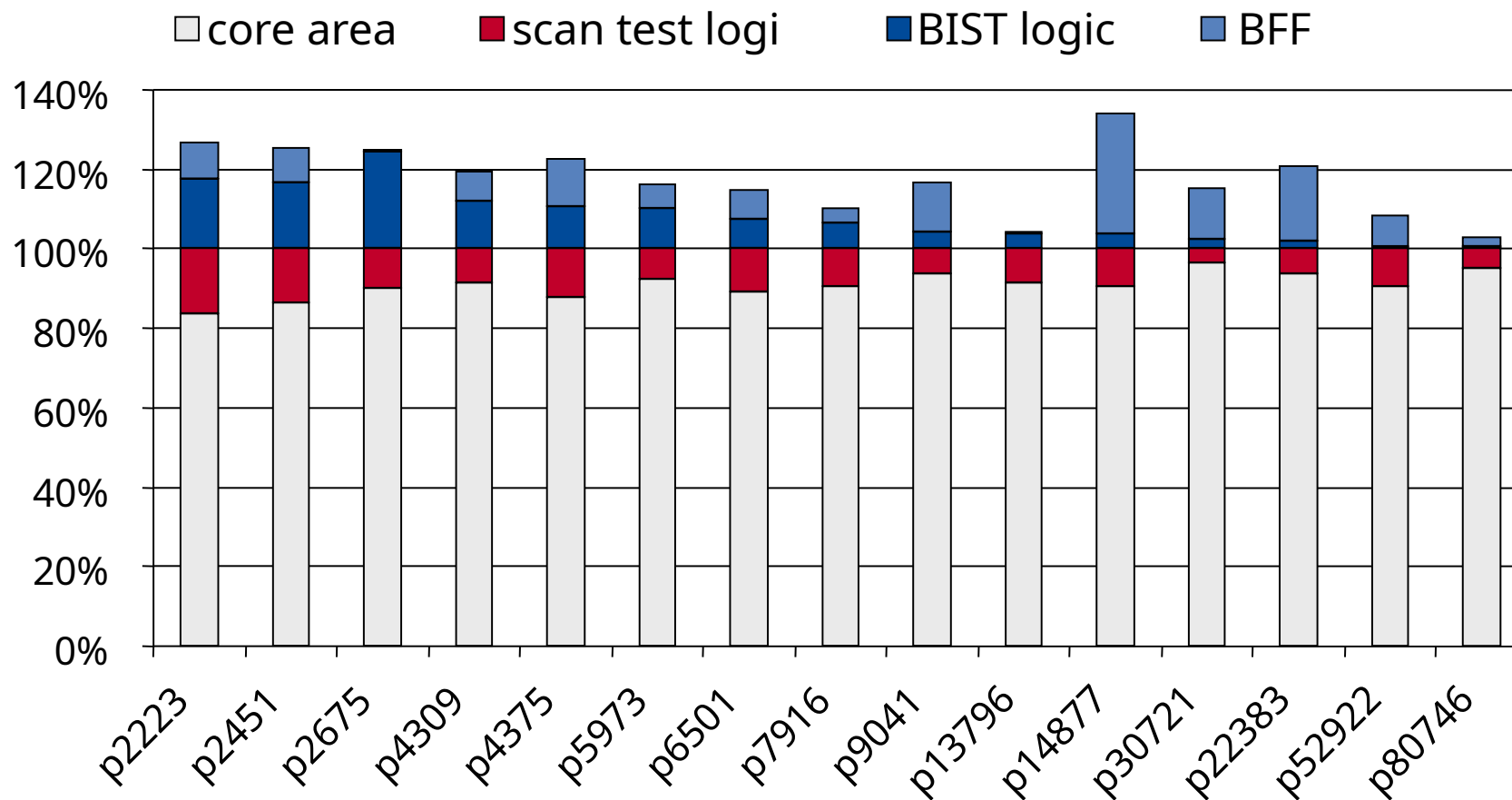
Maximum Number of Specified Bits

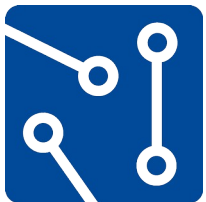




Synthesis Results

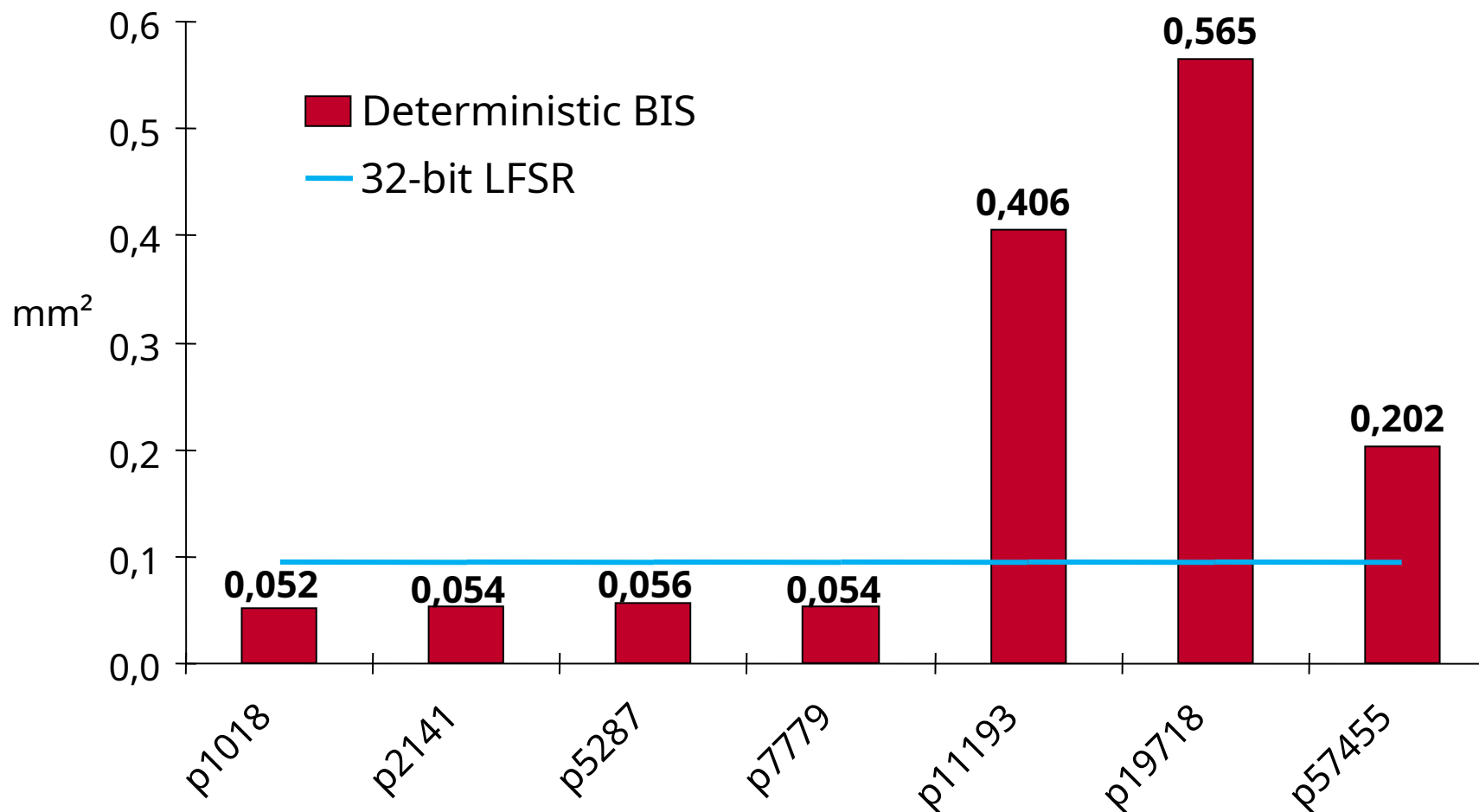
Technology: Philips 0.25μm CMOS

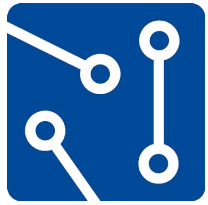




Area of Pattern Generator (Remaining Circuits)

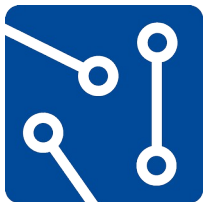
Technology: 1 μ m CMOS, PLA macro cell(s)



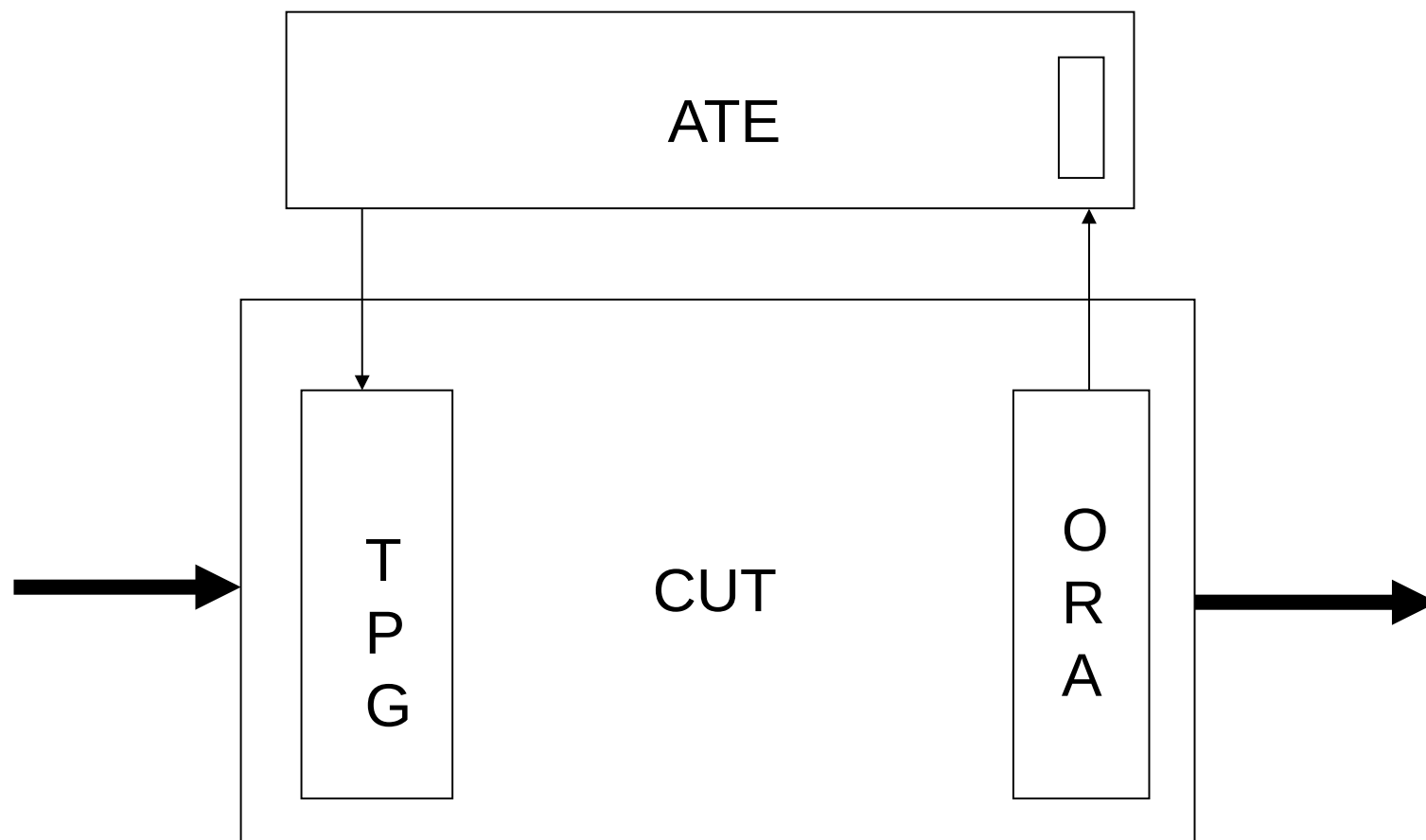


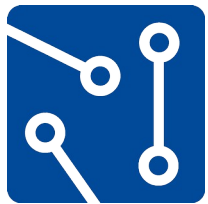
Efficiency of Deterministic BIST

- **High fault coverage** without impact on mission logic.
- Area efficiency like pseudo-random schemes.
- For complete coverage:
 - Less hardware cost than costs of scan path.
- Reduced activity during test:
 - Less power consumption.



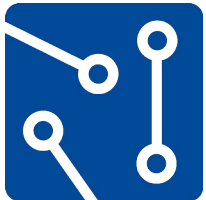
General BIST Structure





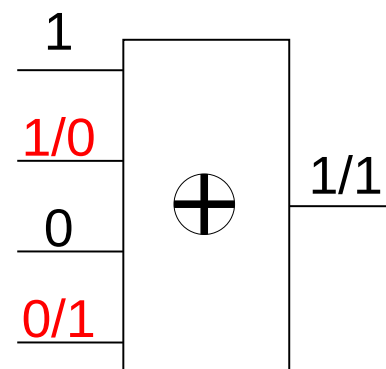
Signature Compaction

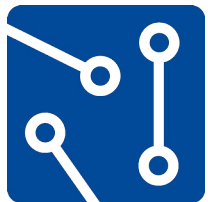
- If K test vectors are applied to a CUT with M outputs, the response consists of $K \times M$ bits
 - Failing CUT is identified by comparing its actual responses to known fault-free responses. Actual responses may be transferred to the tester, or fault-free responses stored on chip
 - M and especially K may be large in BIST, resulting in **impractical memory requirements**
- **Goal:** Reduce the amount of output data that needs to be considered by calculating a **signature** of the actual response and comparing it with a **fault-free signature**



Signature Compactor Design

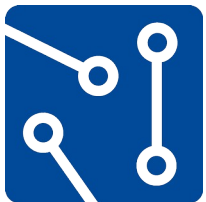
- **Aliasing / Error Masking:**
Loss of fault detection
due to signature compaction
- Goal 1: High compression ratio
- Goal 2: Low error masking
- Goal 3: Low overhead
 - Area, Power, Delay, ...





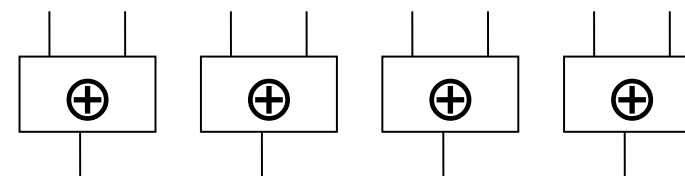
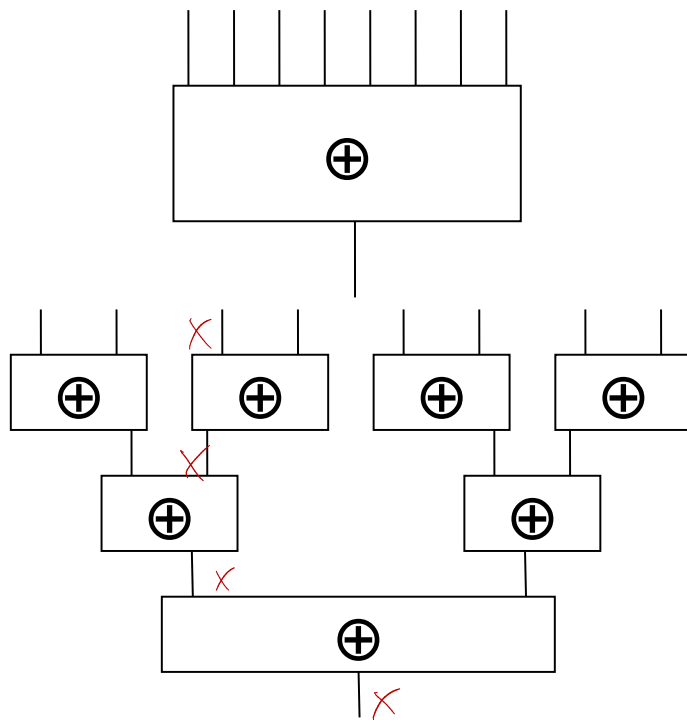
Types of Signature Compaction

- **Space compaction:** Reduce the number M of outputs to be considered
 - E.g. XOR trees
- **Time compaction:** Reduce the number K of vectors to be considered
 - E.g. Counting ones on every output

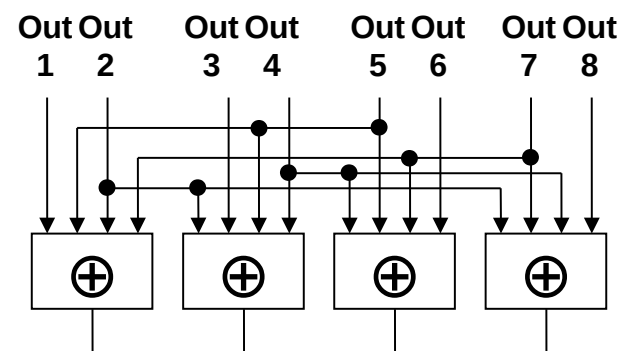


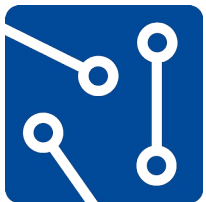
Space compaction

- One XOR gate (or an XOR tree):
 - high compression ratio
 - high aliasing
- Less aliasing: > 1 output



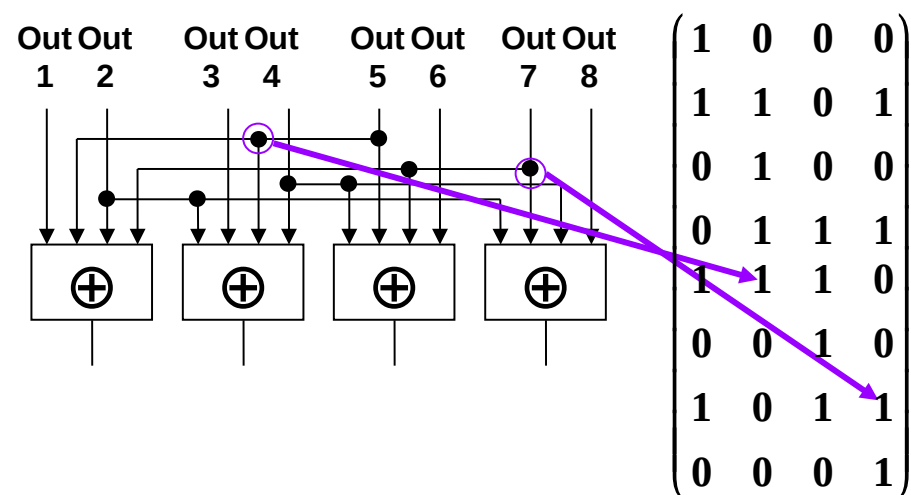
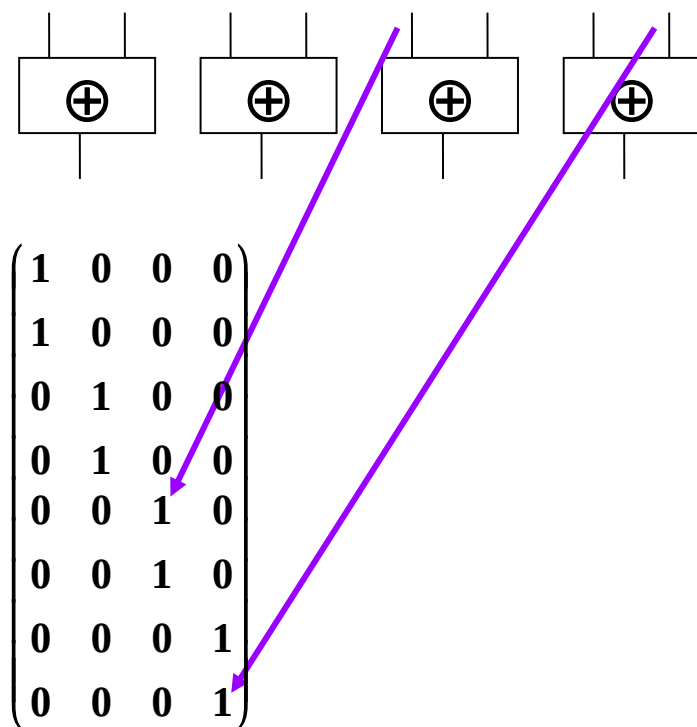
- Better properties with same compression ratio

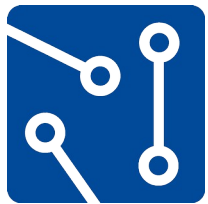




Matrix Representation

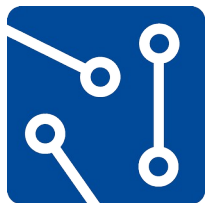
- For a compactor with M inputs (CUT outputs) and P outputs, the matrix C is $(M \times P)$ -dimensional
- $C_{ij} = 1$ if input i connected with output j, 0 otherwise





Properties

- We consider only Cs with non-zero rows, i.e., every input is connected to at least one output
- **Property 1:** An error on a **single** CUT output at any cycle is guaranteed to be detected (produce errors at compactor's output)
- **Property 2:** An error from **any two** outputs at the same cycle are guaranteed to be detected iff all rows are distinct
- **Property 3:** Errors from any **one, two, or odd number** of outputs at the same cycle are guaranteed to be detected iff all rows are distinct and contain an odd number of 1s.



Time Compaction

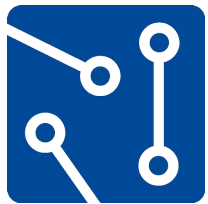
- **Ones count:** For each CUT output, count the number of 1s in all the responses

- Alternatives:

- Count 1 $\rightarrow$ 0, 0 $\rightarrow$ 1 transitions
- Count number of runs

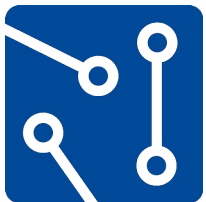
<u>Output #:</u>	<u>1</u>	<u>2</u>	<u>3</u>	<u>4</u>	<u>5</u>	<u>6</u>	<u>7</u>
Response 1:	0	0	1	1	0	0	0
Response 2:	1	1	1	0	0	1	0
Response 3:	1	0	1	1	0	1	1
Sum:	2	1	3	2	0	2	1

<u>Output #:</u>	<u>1</u>	<u>2</u>	<u>3</u>	<u>4</u>	<u>5</u>	<u>6</u>	<u>7</u>
Response 1:	0	1	1	1	0	0	0
Response 2:	1	1	1	1	0	1	0
Response 3:	1	0	1	0	0	1	1
Sum:	2	2	3	2	0	2	1



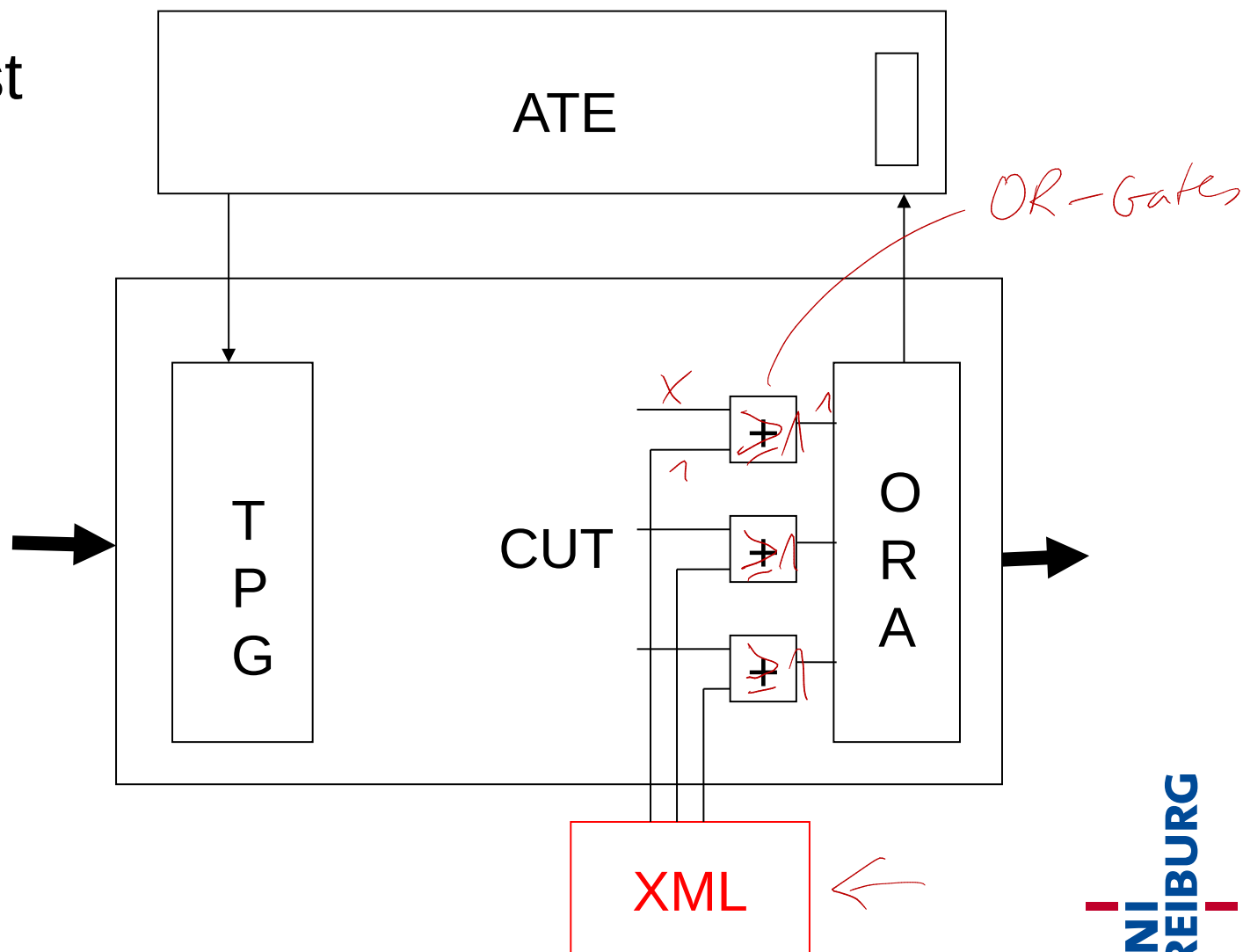
Handling Unknowns

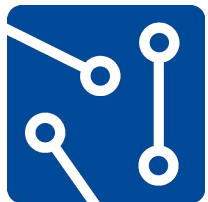
- Some circuits produce **unknowns** (U, X)
 - Floating buses, uninitialized flipflops, clock domains, analog parts, Xs at the inputs
- In space compaction, a X on one CUT output **invalidates** the compactor output it is connected to, i.e., all detections on this output are lost
- In time compaction, one X at one output may invalidate the **whole signature**
- **Solutions:** X **elimination**, X-**tolerant** compactors



X Elimination: X Masking Logic (XML)

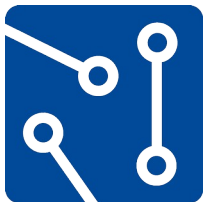
- Find out by simulation test vectors/outputs where unknowns show up
- Synthesize block that produces a 1 for these vectors/outputs, 0 otherwise





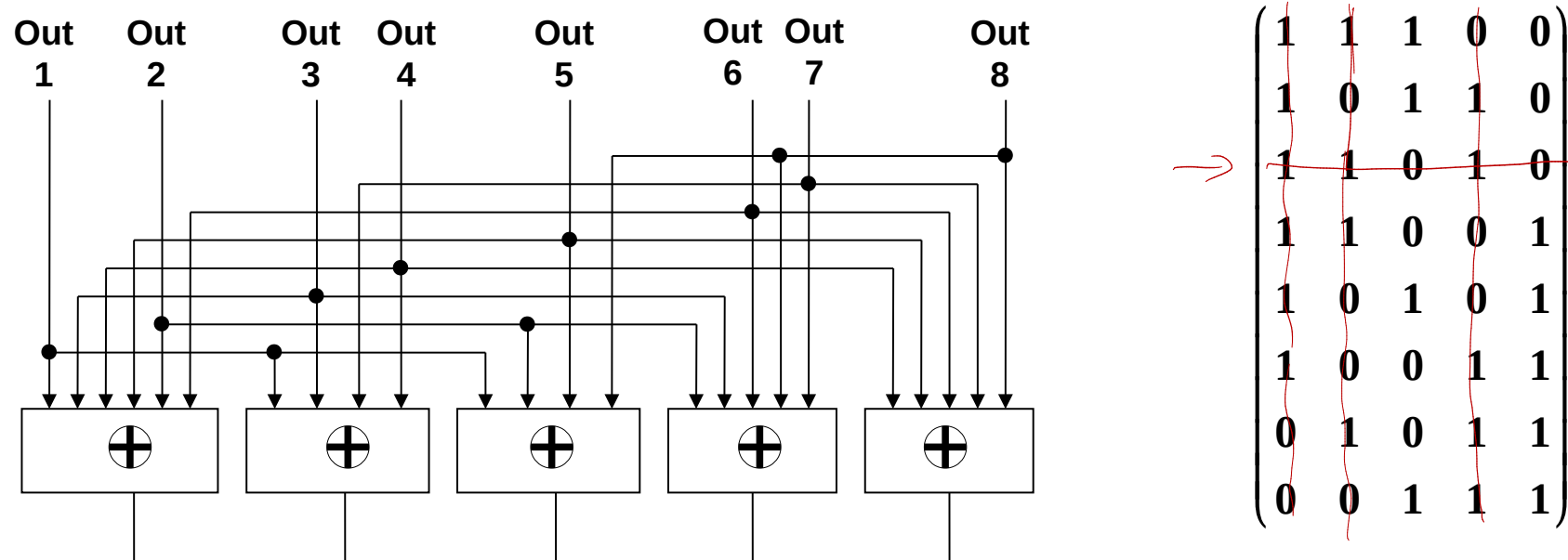
X Masking Logic: Drawbacks

- Needs to be **re-synthesized** when test vectors change
 - Regular structures, e.g. LFSR with reseeding, can be used
- Logic may become too large
 - Allow blocking for vectors/outputs without Us
 - Potential coverage loss
 - Ensure that all stuck-at faults are still detected
 - Coverage loss of unmodeled defects

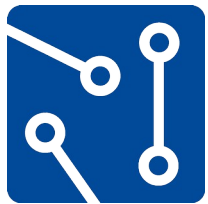


X-tolerant compactor: X-Compact

Mitra

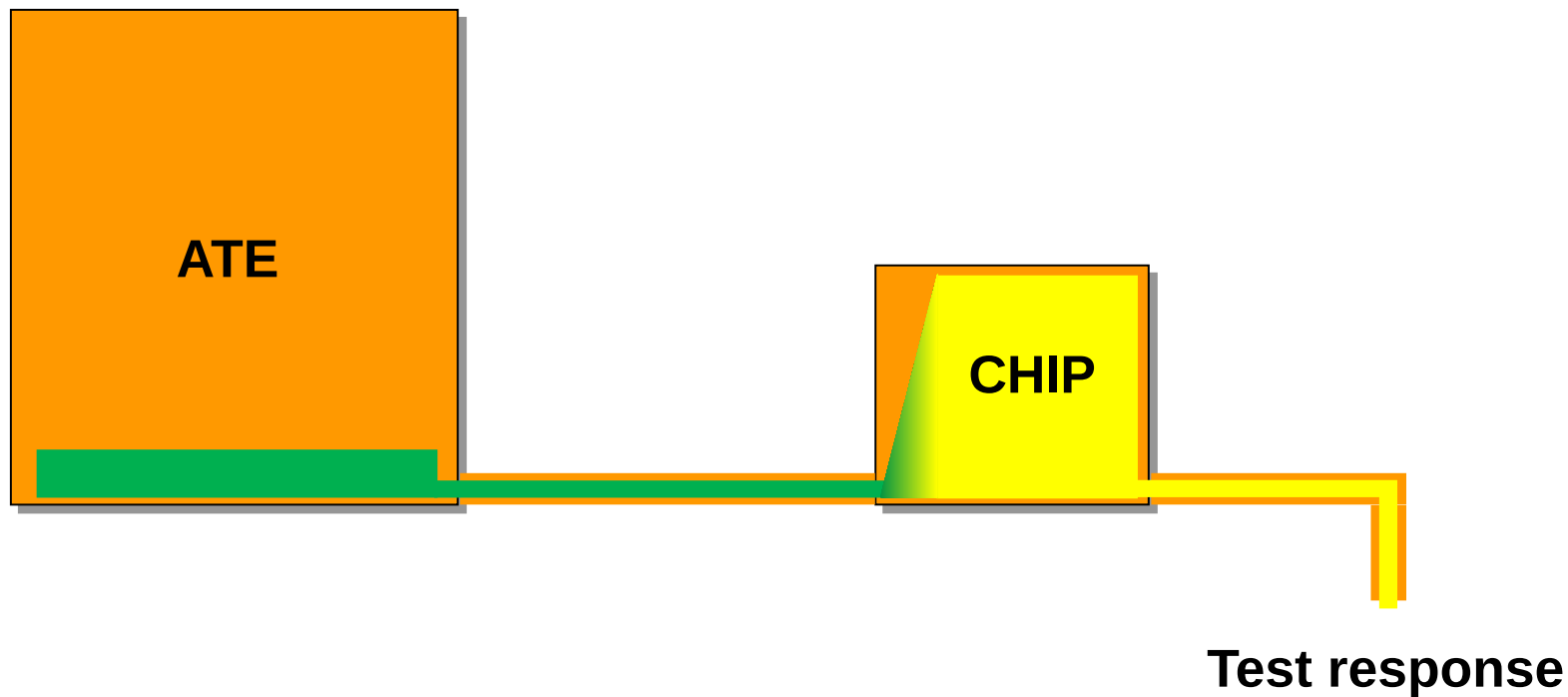


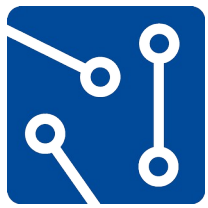
- **Property 4:** An error on any single CUT output and an X from any other single CUT output at the same cycle are guaranteed to be detected iff for any matrix row, the submatrix obtained by removing that row and all the columns having 1 in that row does not contain a row with all 0s
- **Corollary:** Properties 3 + 4 are satisfied if every row is distinct, has an equal and odd number of 1



Test Compression

- High Test data volume
- Limited bandwidth ATE – Chip:
 - High Test application time



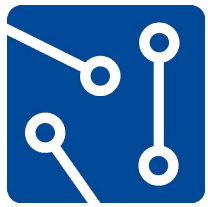


„Properties“ of Test Data

■ e.g.

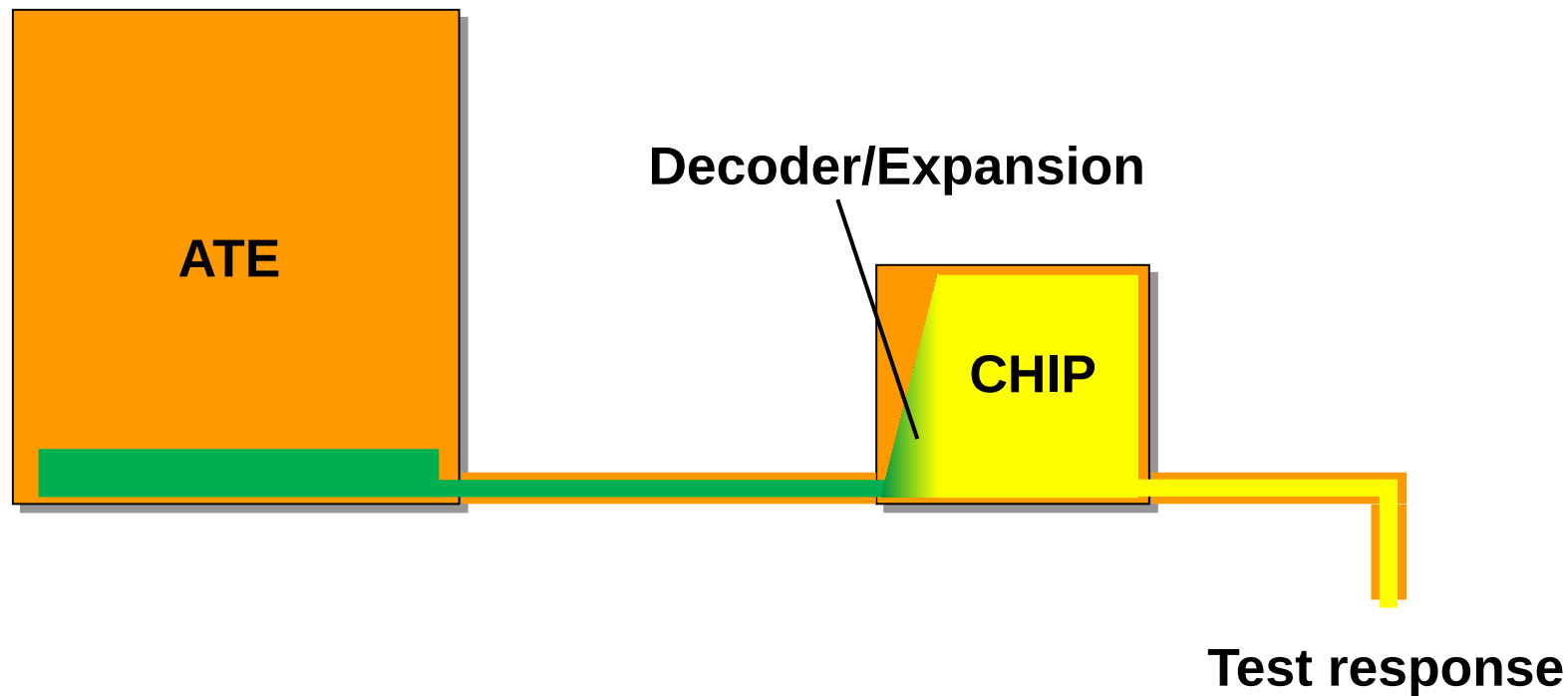
```
10-----0---1
10-----0---0
11-----0---1
10-----11---1---1
10-----0---0---1---1
10-----1---001---1---
```

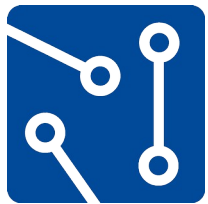
- >95% : don't cares
- >95% of test time: transport of don't cares



Test Data Compression

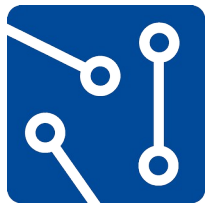
- Save test data in coded form
- Decode/Expand test data on-chip





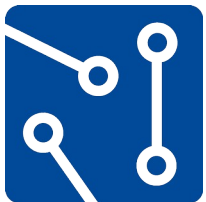
Continuous-flow Test Compression

- Insert a large number of internal scan chains (more than there are I/O pins)
- On-chip decompressor continuously receives compressed data through the I/O pins and drives the internal scan chains
- If there are 5 input pins and 100 internal chains, the **compression ratio** is 20X
 - In one clock cycle, 5 bits are communicated to the chip and 100 bits of a test are generated



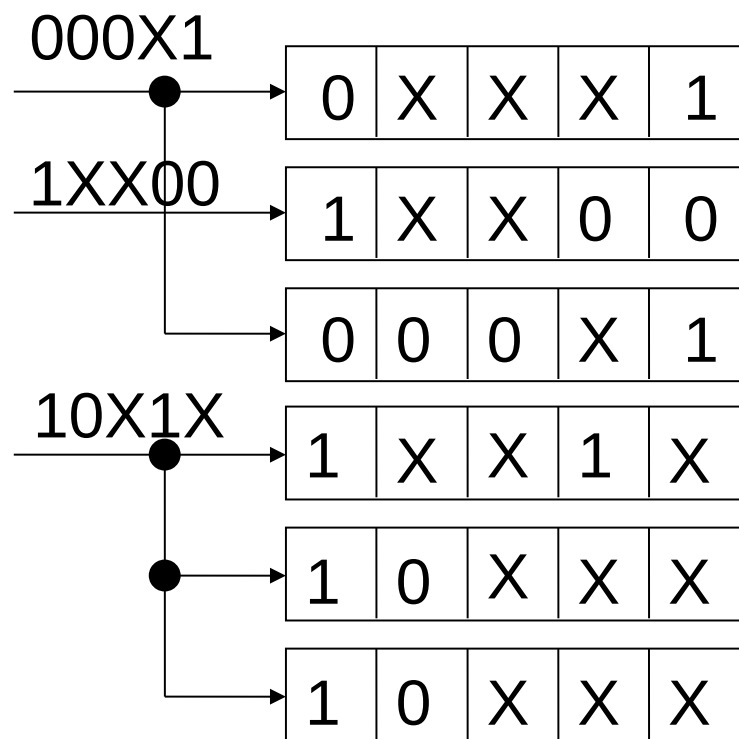
Examples Used in Practice

- **Illinois scan:** feed same test data to multiple internal scan chains
 - Compressed data obtained by compatibility analysis
- **EDT (Mentor Graphics):** use a linear decoder similar to LFSR
 - Compressed data is obtained by solving a system of linear equations
- In both cases, achievable compression ratio depends on the number of don't cares and their positions in the test data

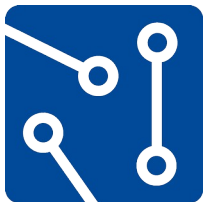


Illinois Scan

- Feed **same data** into different scan chains

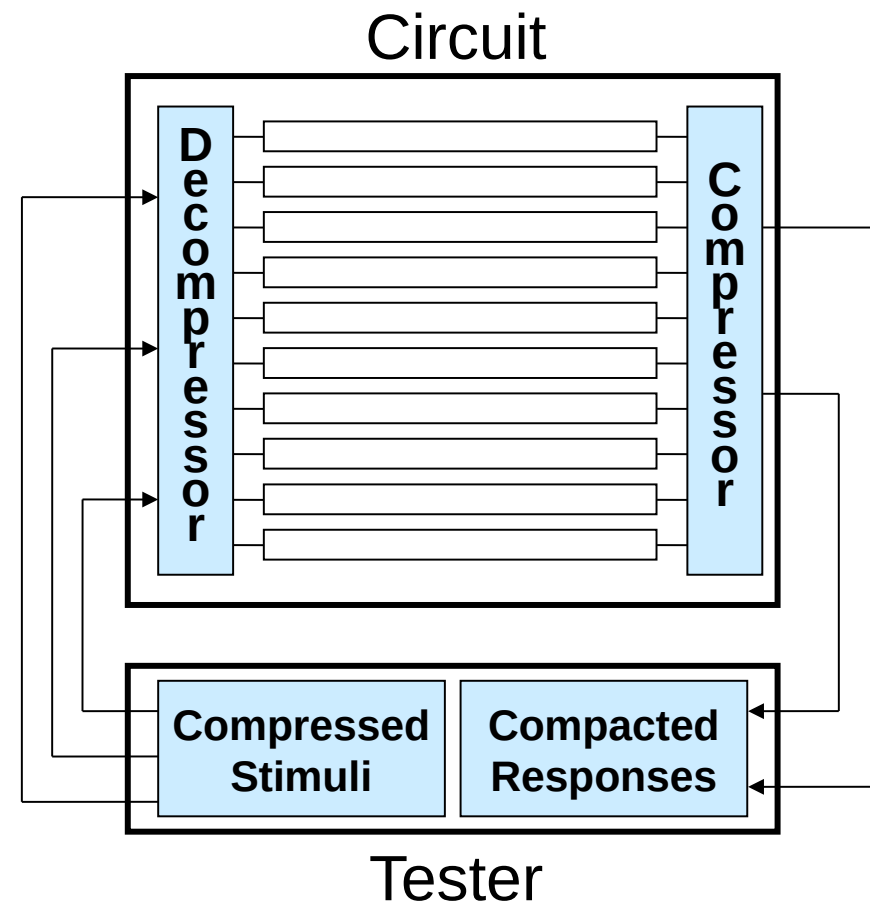


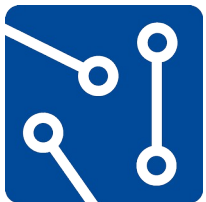
- Multiplex an input to several scan chains
- Compression ratio = $\frac{\text{\#scan chains}}{\text{\#inputs}}$
 - e.g. 6 chains, 3 inputs: compression ratio = 2X
 - In the example, 15 bits are provided by the tester to obtain 30 bits in scan chains



Embedded Deterministic Test (EDT)

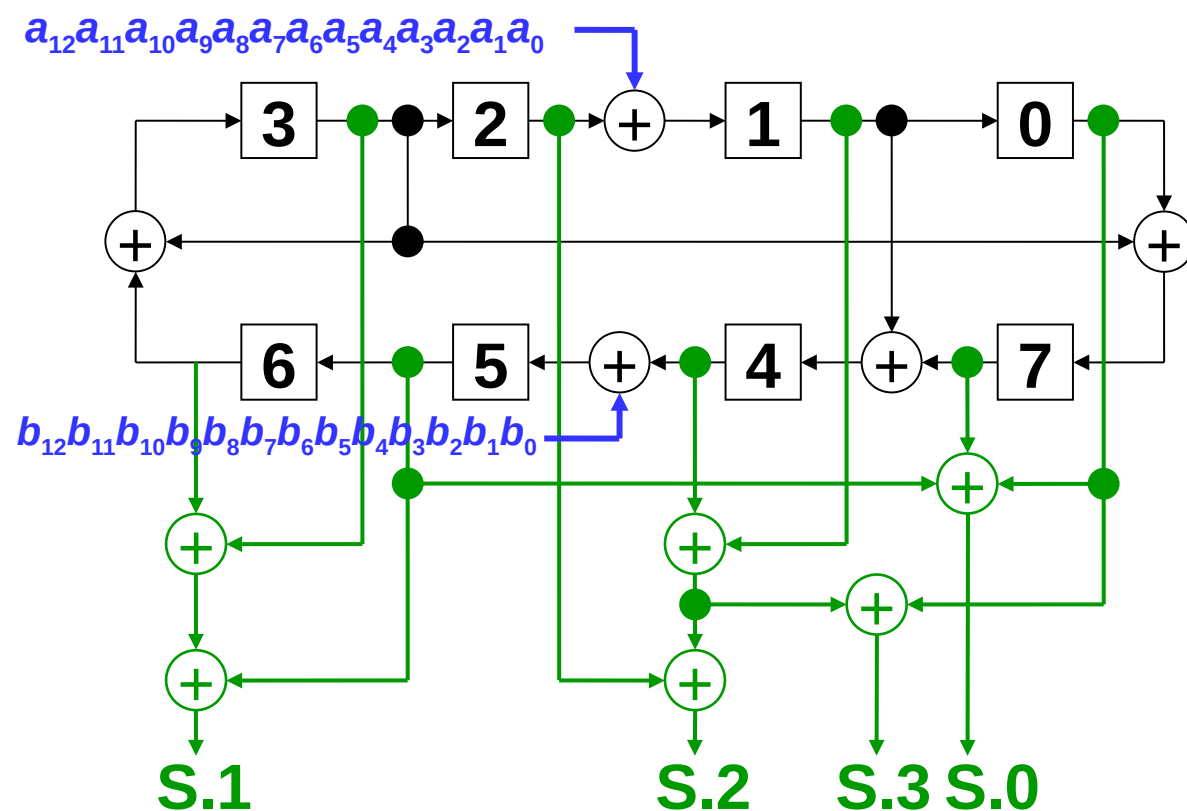
- Apply tests to a CUT with many scan chains using few tester channels and IC pins
 - E.g. 500 chains, 20 channels: 25X compression

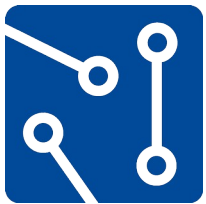




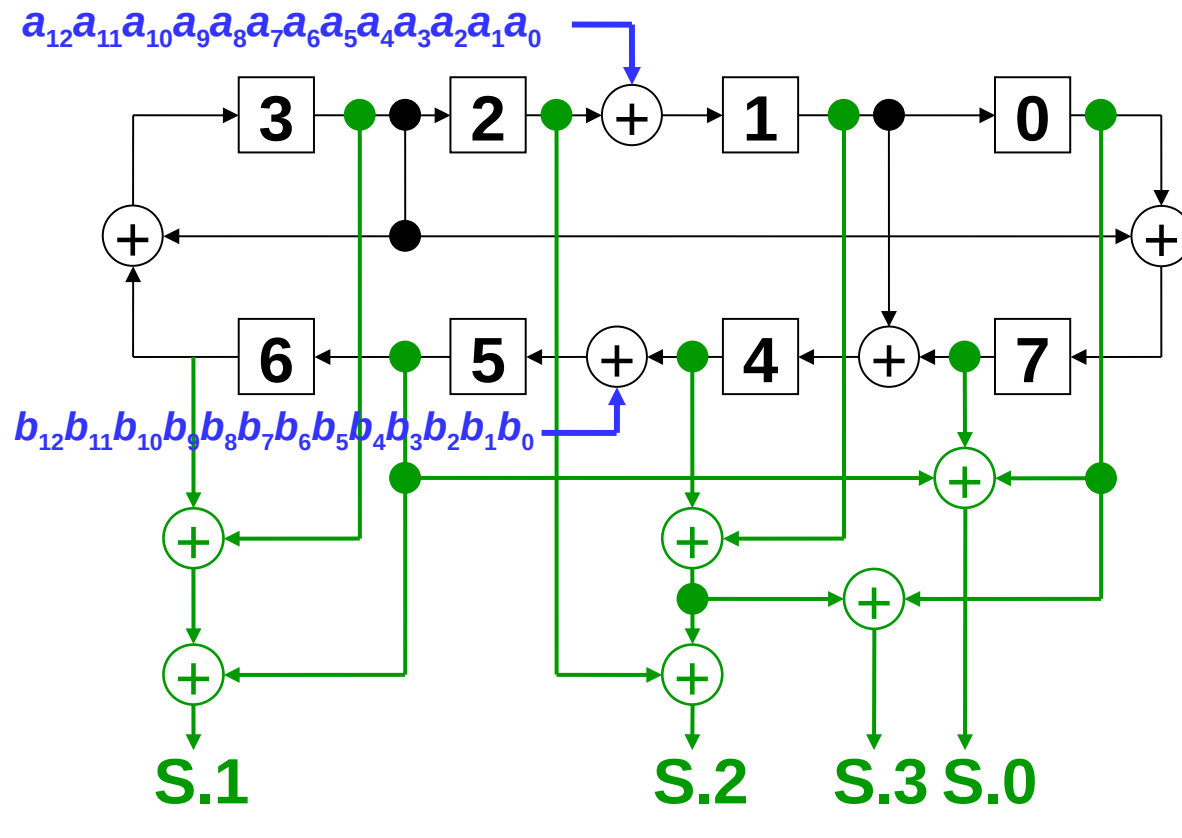
Continuous Flow Decompressor

- Two input streams (a , b)
- Four outputs which drive four scan chains ($S.0$, $S.1$, $S.2$, $S.3$)
- Eight internal flip-flops (0 through 7)
- Values in flip-flops and at outputs are functions of a and b



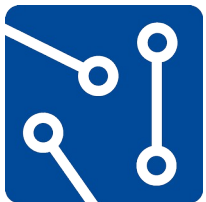


Continuous Flow Decompressor

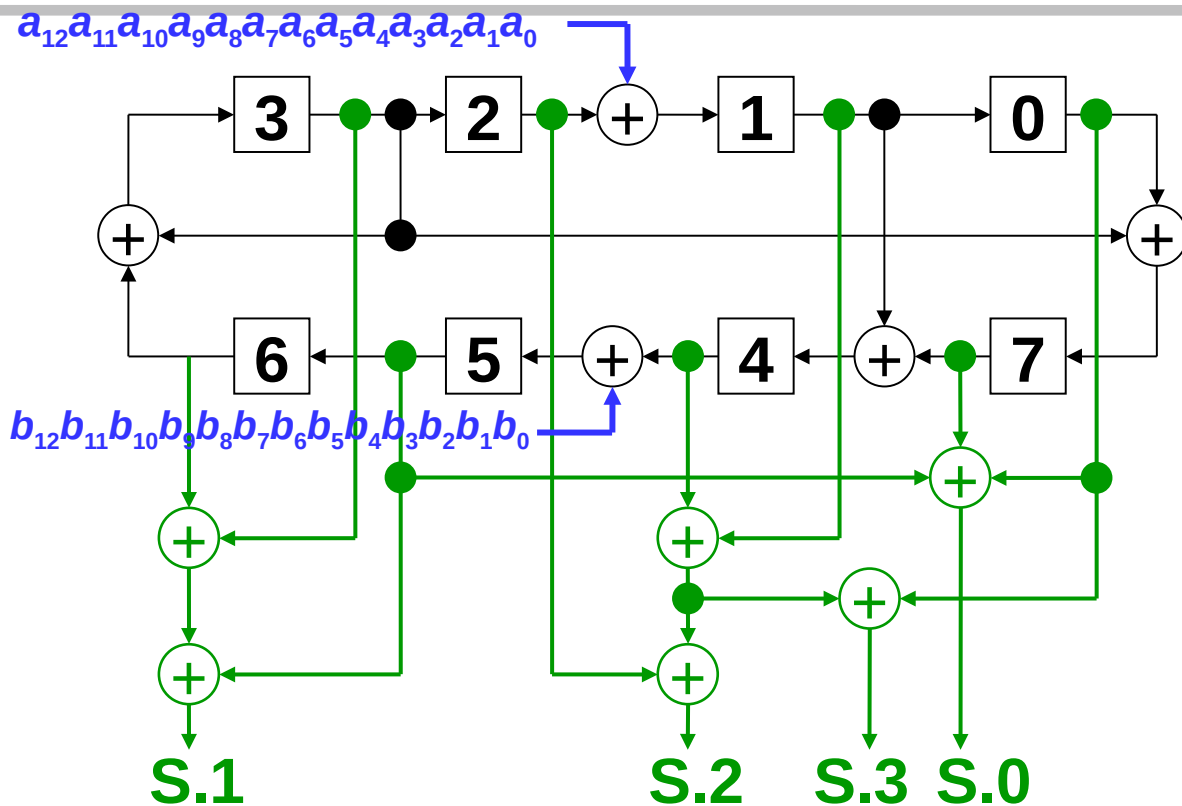


0	1	2
0	a_0	0
a_0	a_1	0
a_1	a_2	0
a_2	a_3	b_0
a_3	$a_4 \oplus b_0$	$b_1 \oplus b_0$
$a_4 \oplus b_0$	$a_5 \oplus b_1 \oplus b_0$	$a_0 \oplus b_2 \oplus b_1 \oplus b_0$
$a_5 \oplus b_1 \oplus b_0$	$a_6 \oplus a_0 \oplus b_2 \oplus b_1 \oplus b_0$	$a_1 \oplus a_0 \oplus b_3 \oplus b_2 \oplus b_1 \oplus b_0$

- After four cycles, values in all flip-flops are determined by a and b
- Values in flip-flops 0, 1, 2 as function of a and b are shown
- From values of flip-flops 0 through 7, the values at outputs ($S.0$, $S.1$, $S.2$, $S.3$) fed into circuit's scan chains can be calculated



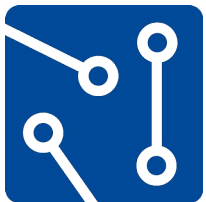
Continuous Flow Decompressor



start feeding

0	1	2
0	a_0	0
a_0	a_1	0
a_1	a_2	0
a_2	a_3	b_0
a_3	$a_4 \oplus b_0$	$b_1 \oplus b_0$
$a_4 \oplus b_0$	$a_5 \oplus b_1 \oplus b_0$	$a_0 \oplus b_2 \oplus b_1 \oplus b_0$
$a_5 \oplus b_1 \oplus b_0$	$a_6 \oplus a_0 \oplus b_2 \oplus b_1 \oplus b_0$	$a_1 \oplus a_0 \oplus b_3 \oplus b_2 \oplus b_1 \oplus b_0$

	S.0	S.1	S.2	S.3
cycle 1	$a_3 \oplus a_0 \oplus b_4 \oplus b_1 \oplus b_0$	$a_3 \oplus a_2 \oplus a_1 \oplus b_4 \oplus b_2 \oplus b_1$	$a_4 \oplus a_1 \oplus b_3 \oplus b_1$	$a_4 \oplus a_3 \oplus a_1 \oplus b_3 \oplus b_0$
cycle 2	$a_4 \oplus a_1 \oplus a_0 \oplus b_5 \oplus b_2 \oplus b_1 \oplus b_0$	$a_4 \oplus a_3 \oplus a_2 \oplus a_0 \oplus b_5 \oplus b_3 \oplus b_2$	$a_5 \oplus a_2 \oplus b_4 \oplus b_2$	$a_5 \oplus a_4 \oplus a_2 \oplus b_4 \oplus b_1$
cycle 3	$a_5 \oplus a_2 \oplus a_1 \oplus a_0 \oplus b_6 \oplus b_3 \oplus b_2 \oplus b_1$	$a_5 \oplus a_4 \oplus a_3 \oplus a_1 \oplus a_0 \oplus b_6 \oplus b_4 \oplus b_3 \oplus b_0$	$a_6 \oplus a_3 \oplus b_5 \oplus b_3 \oplus b_0$	$a_6 \oplus a_5 \oplus a_3 \oplus a_1 \oplus a_0 \oplus b_5 \oplus b_2 \oplus b_0$



Compression of a Test Vector

	9	8	7	6	5	4	3	2	1
S.0	X	X	X	X	X	X	1	X	1
S.1	1	X	X	0	X	1	1	X	X
S.2	X	X	X	X	X	X	X	X	X
S.3	0	0	X	X	1	X	X	0	X

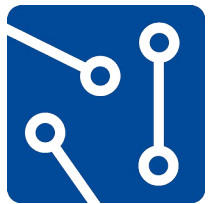
$$a_3 \oplus a_0 \oplus b_4 \oplus b_1 \oplus b_0 = 1$$

$$a_5 \oplus a_4 \oplus a_2 \oplus a_0 \oplus b_4 \oplus b_1 = 0$$

$$a_5 \oplus a_2 \oplus a_1 \oplus a_0 \oplus b_6 \oplus b_3 \oplus b_2 \oplus b_1 = 1$$

etc.

- Every specified bit corresponds to a linear equation (3 equations shown)
- Linear Equation System (LES) solution: $a_2=a_1=b_6=b_4=b_1=1$, others=0
- In dynamic compaction, more bit positions may get specified.



Compression of a Vector Set

```
while (not all faults targeted)
  select fault and generate cube (vector with Xes)
  try to compress cube
  if (compression failed) declare fault as EDT aborted
  loop
    select fault; generate incremental test cube
    if (dynamic compaction failed) continue with new fault
    try to compress cube incrementally
    if (compression failed) continue with new fault
  end loop
end while
```